

Researcher trends in quantum computing

SE-UIT-VNUHCM, 19-Dec-2025

PhD. Vu Tuan Hai

Email: haivt@uit.edu.vn

Outline

- 0. Quantum foundation
- 1. Quantum optimization
- 2. Quantum machine learning

0. Quantum foundation

0.1. Models (quantum circuit, adiabatic, annealing, ...)

0.2. Complexity (BQP)

0.3. Quantum information (entanglement, quantum channel, noise)

0.4. Quantum algorithms

0.5. ECC

0.6. Quantum computing for physical problems

1. Quantum optimization

1.1. Parameterized quantum circuit (PQC)

1.2. Circuit ansatz

1.3. Compilation

1.4. Gradient and optimizer

1.5. Encoding

1.6. Quantum optimization

1.7. Additional informations

1.1. Parameterized quantum circuit (PQC)

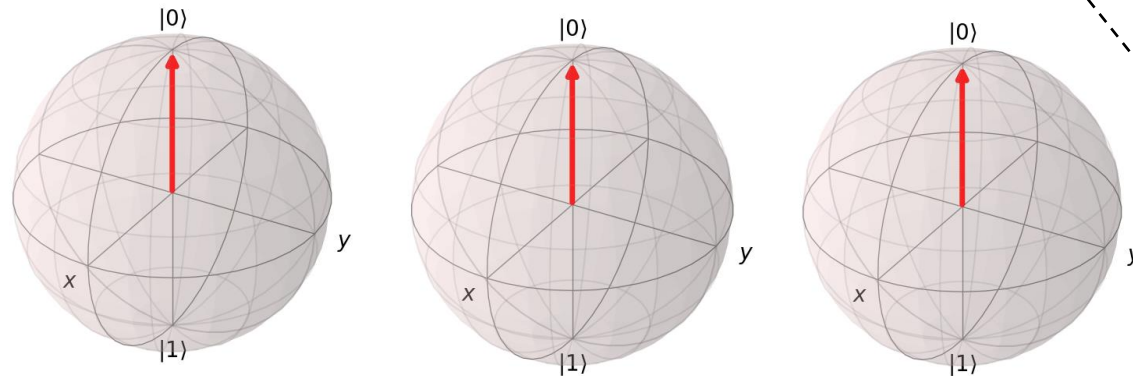
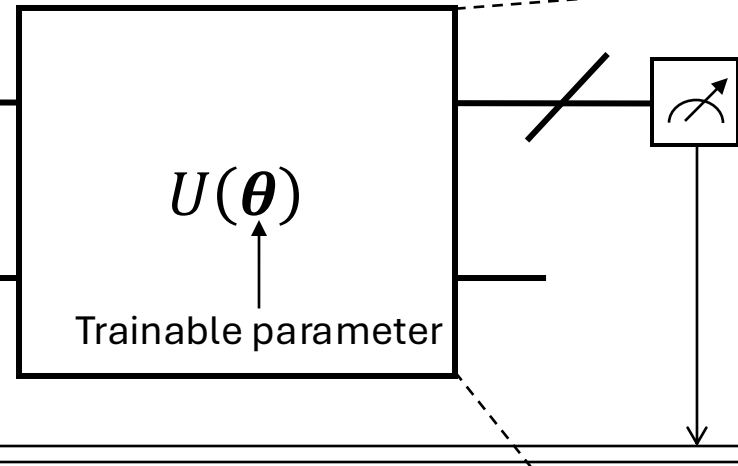
(Main) qubits

$|0\rangle^{\otimes m}$

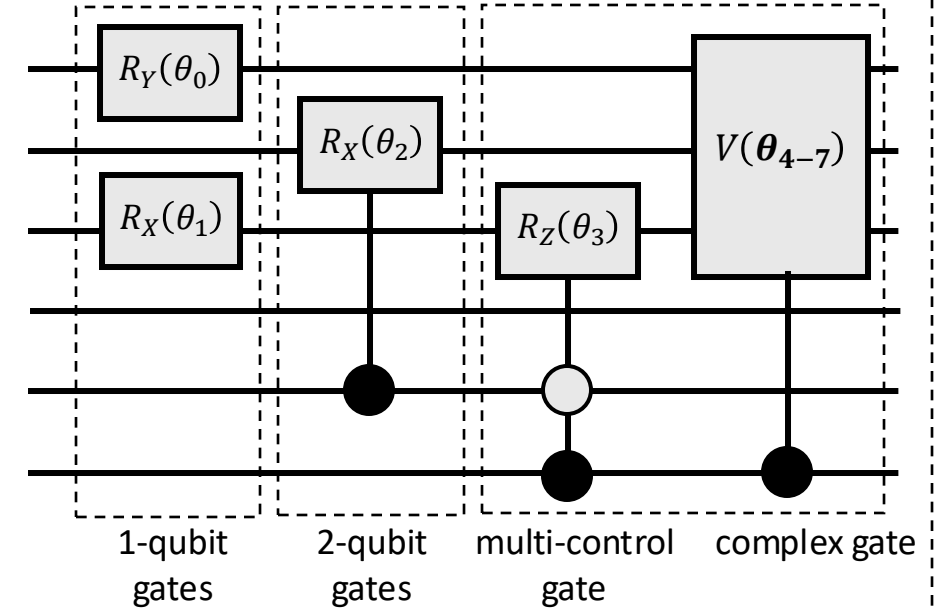
Ancilla qubits

$|0\rangle^{\otimes n}$

Classical
channel



Gate $R_i(\theta)$ rotate state θ^0 about the i -axis
where $i \in \{X, Y, Z\}$ and $\theta \in [0, 2\pi]$

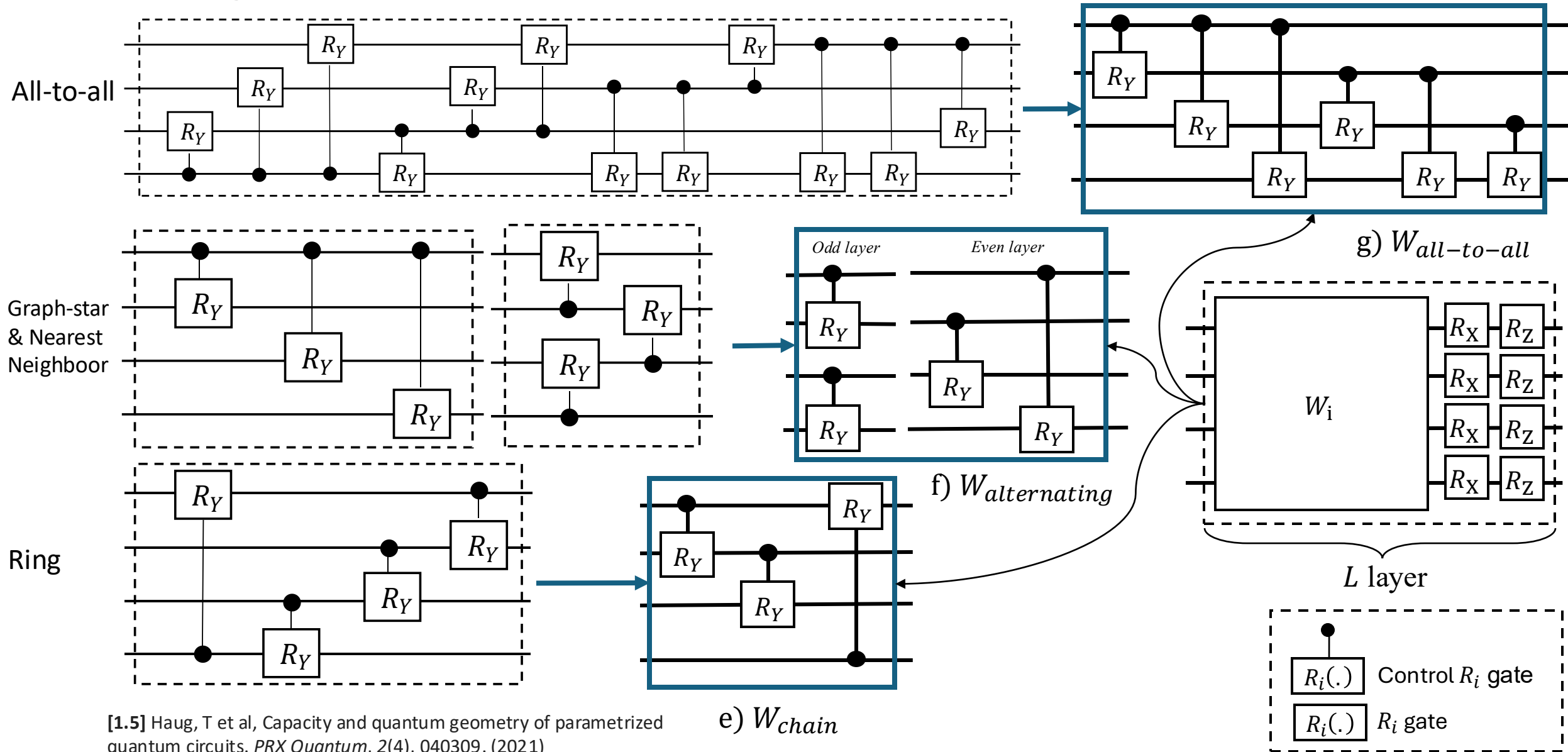


$$\theta = [\theta_0 \quad \theta_1 \quad \dots]^T$$

[1.2] Benedetti, M et al, Parameterized quantum circuits as machine learning models. Quantum Science and Technology, 4(4), 043001 (2019)

1.2. Circuit ansatz

Existing ansatzes



1.4. Gradient and optimizer

Considering a PQC:

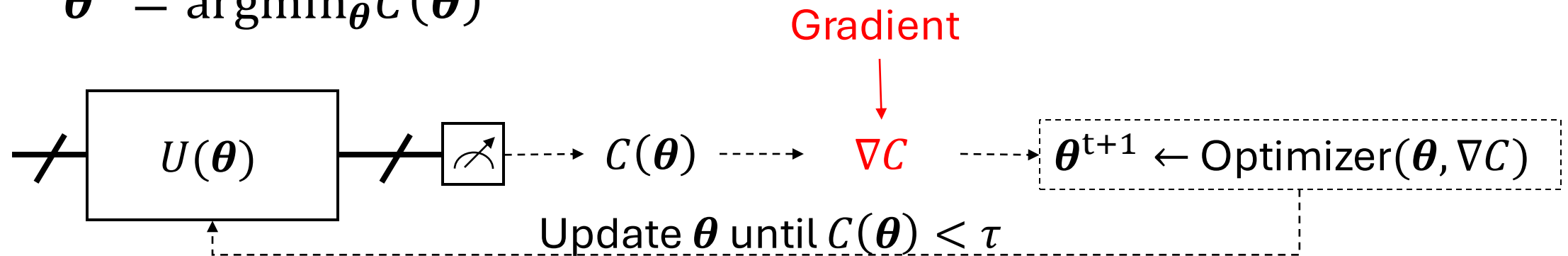
$$U(\boldsymbol{\theta}) = U_{m-1}U_{m-2} \dots U_0$$

can be splitted into m sub-circuits with $|\boldsymbol{\theta}| = m$
and cost function:

$$C(\boldsymbol{\theta}) = \langle \mathbf{0} | U^\dagger(\boldsymbol{\theta}) \hat{B} U(\boldsymbol{\theta}) | \mathbf{0} \rangle$$

We want to find:

$$\boldsymbol{\theta}^* = \operatorname{argmin}_{\boldsymbol{\theta}} C(\boldsymbol{\theta})$$



1.4. Gradient

Computing PQC's gradient is the key in quantum optimization:

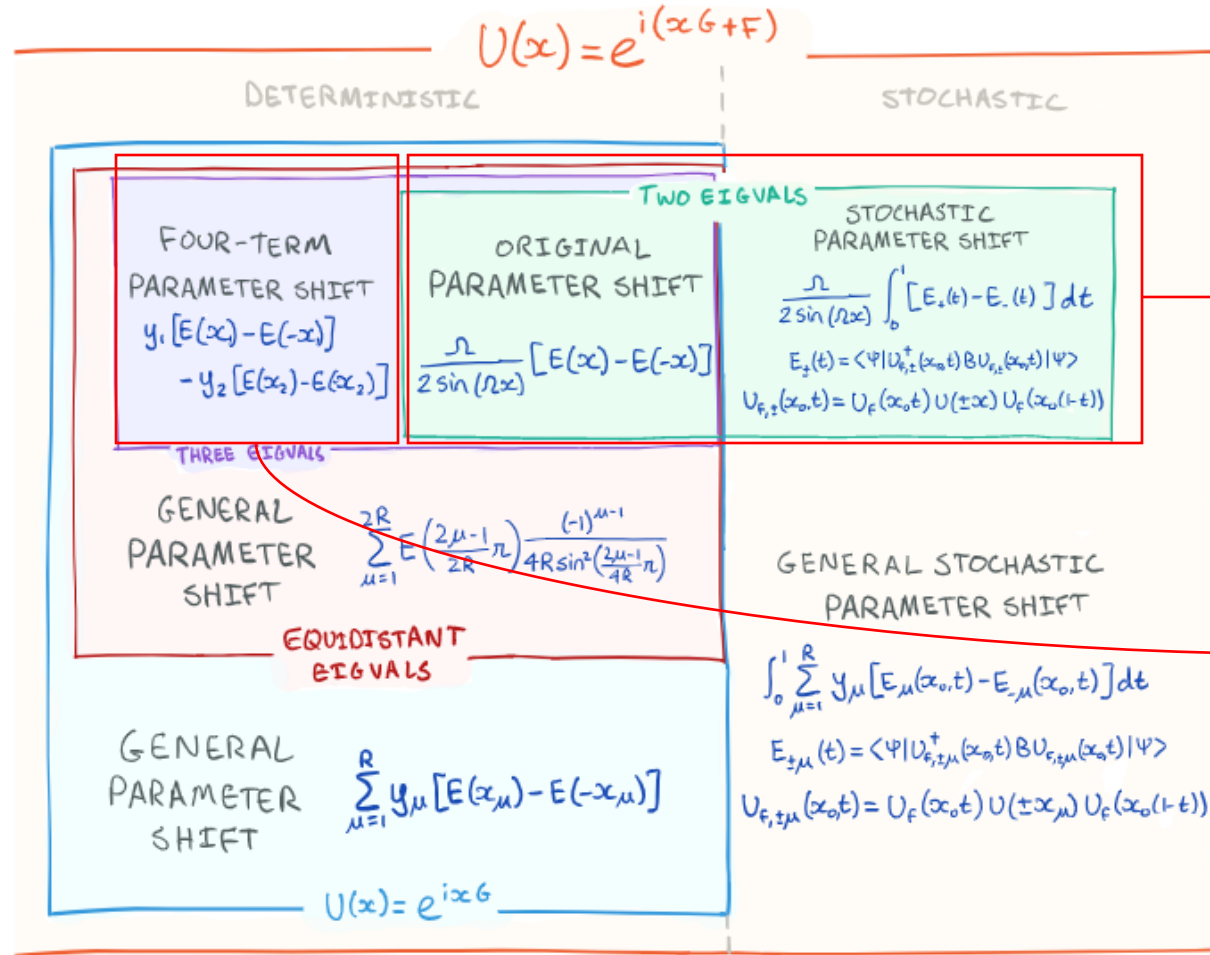
$$\nabla C = [\partial_{\theta_0} C \quad \dots \quad \partial_{\theta_{m-1}} C]^\top$$

The partial derivative can be computed through parameter-shift rule **[1.17]**

$$\frac{\partial C}{\partial \theta_j} = \sum_{k=1}^R d_k \left(C(\theta_j^{+\epsilon_k}) - C(\theta_j^{-\epsilon_k}) \right)$$

and $C(\theta_j^{+\epsilon_k}) \equiv C(\boldsymbol{\theta} + \mathbf{e}_j \epsilon_k)$, $\mathbf{e}_j$ is the j^{th} unit vector, ϵ_k is the shift-value

1.4. Gradient Parameter-shift rule



Commonly, we only use 2term-PSR, in case R_i .

$$\partial_{\theta_j} C = \frac{1}{2} \left[C \left(\theta_j^{+\frac{\pi}{2}} \right) - C \left(\theta_j^{-\frac{\pi}{2}} \right) \right]$$

4term-PSR is used in case control- R_i .

$$\partial_{\theta_j} C = d^+ [C(\theta_j^{+\alpha}) - C(\theta_j^{-\alpha})] - d^- [C(\theta_j^{+\beta}) - C(\theta_j^{-\beta})]$$

Take $2m/4m$ quantum evaluations for a single gradient!

$$f(x) = \sin x, f'(x) = \frac{1}{2} \left[\sin \left(x + \frac{\pi}{2} \right) - \sin \left(x - \frac{\pi}{2} \right) \right] = \frac{1}{2} [\cos(x) + \cos(x)] = \cos(x)$$

1.6. Quantum optimization

Considering a **PQC**:

$$U(\boldsymbol{\theta}) = U_{m-1}U_{m-2} \dots U_0$$

can be splitted into m sub-circuit with $|\boldsymbol{\theta}| = m$ and **cost function**:

$$C(\boldsymbol{\theta}) = \langle \mathbf{0} | U^\dagger(\boldsymbol{\theta}) \hat{B} U(\boldsymbol{\theta}) | \mathbf{0} \rangle$$

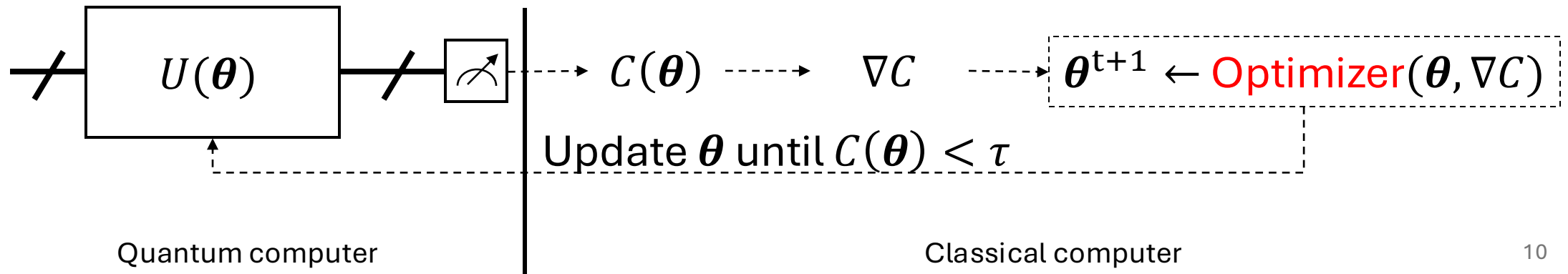
Trainable parameters

Objective associated with expectation values

We want to find:

$$\boldsymbol{\theta}^* = \operatorname{argmin}_{\boldsymbol{\theta}} C(\boldsymbol{\theta})$$

Optimal parameters

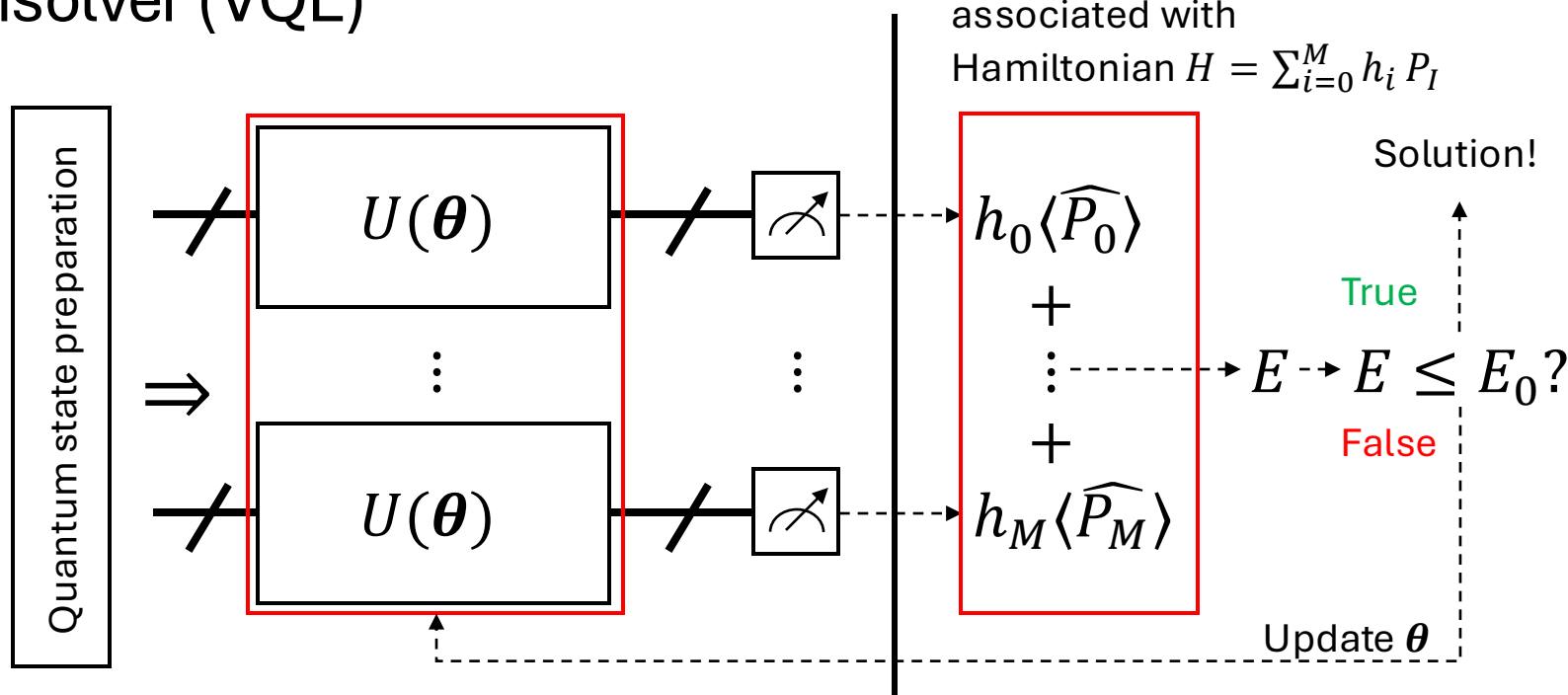
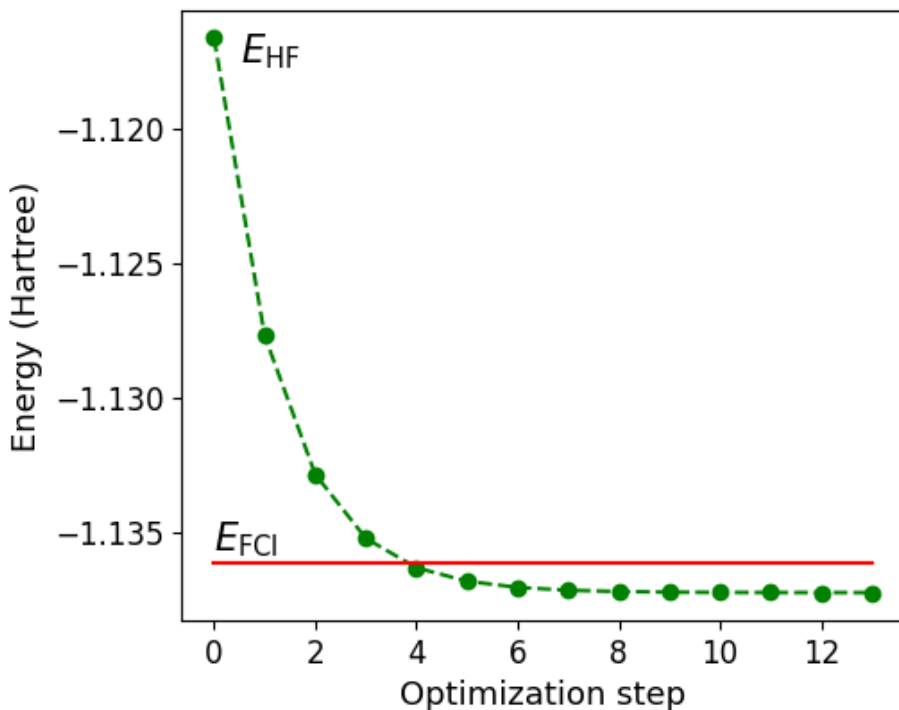


1.6.1. Quantum optimization

Variational Quantum Eigensolver (VQE)

Finding ground energy E_0 corresponding to ground state $|\psi_0\rangle$ of Hamiltonian H

$$H|\psi_0\rangle = E_0|\psi_0\rangle$$



Simple Hamiltonian: $\hat{H}_{Ising} = -\sum_i J_i Z_i Z_{i+1} - \sum_i h_i X_i$

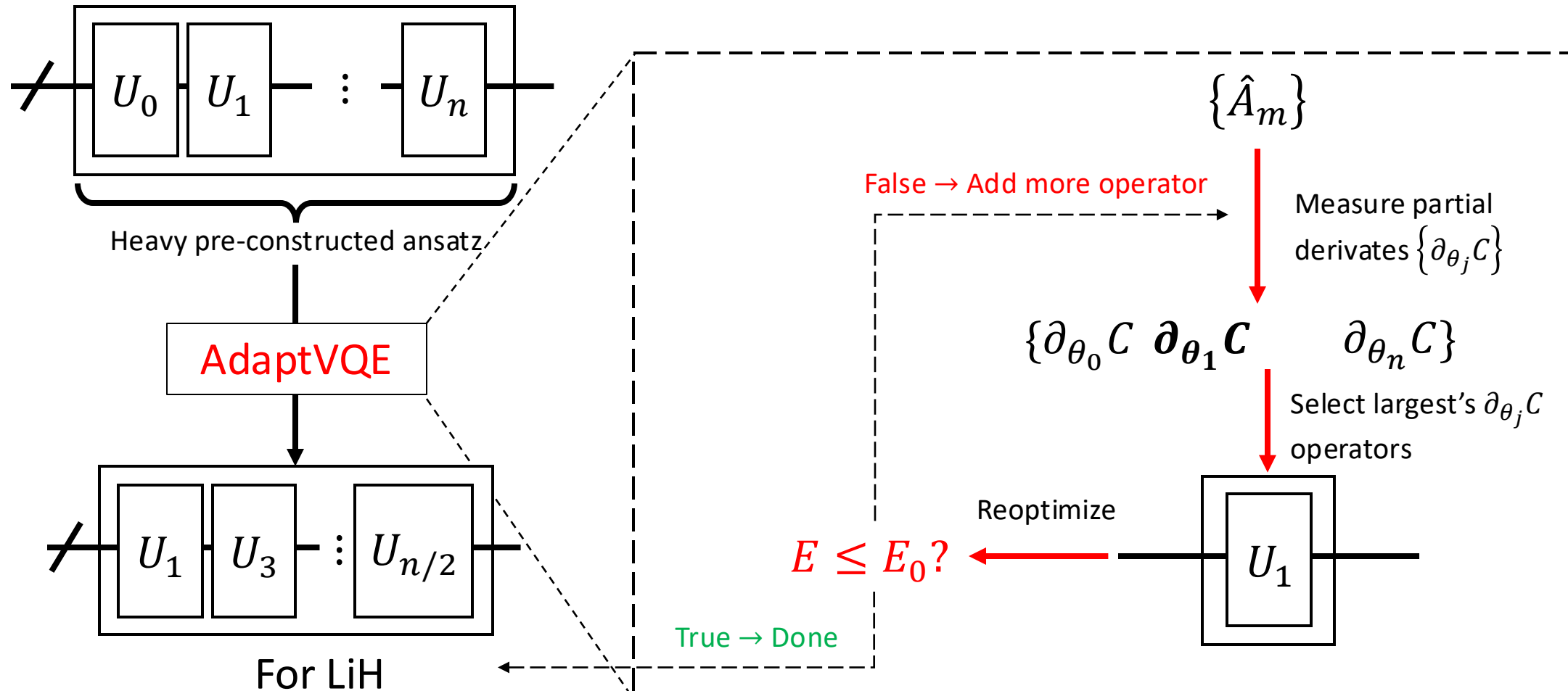
Electronic structure $\rightarrow$ Basis set expansion (ex: sto3g)

$\rightarrow$ Mapping to qubits (ex: JW transform)

$\rightarrow$ Pauli decomposition $\rightarrow \hat{H}$

1.6.1. Quantum optimization

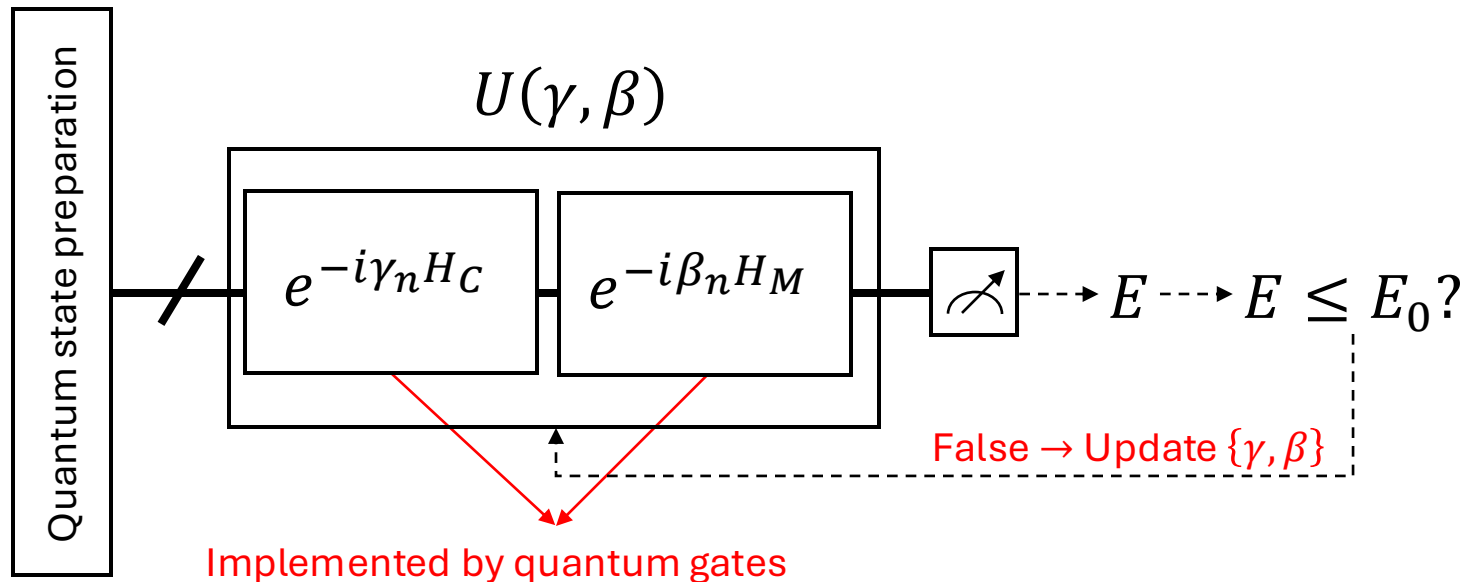
VQE → AdaptVQE



[1.32] Grimsley, H. R., Economou, S. E., Barnes, E., & Mayhall, N. J. (2019). An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nature communications*, 10(1), 3007.

1.6.2. Quantum optimization

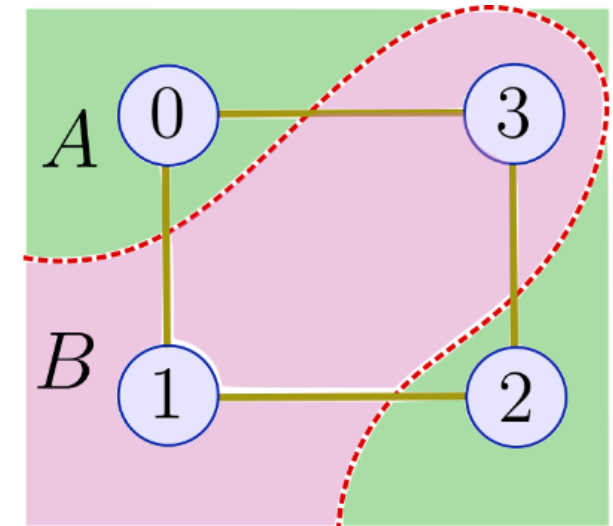
Quantum Approximate Optimization Algorithm (QAOA)



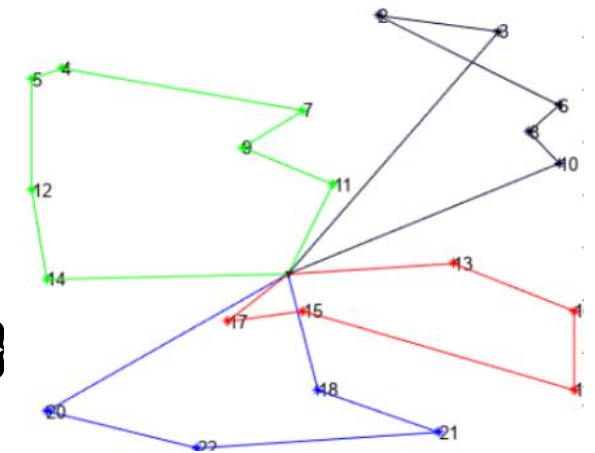
1. Problem Definition (QUBO)

2. Ising Hamiltonian: $\hat{H}_{Ising} = -\sum_i J_i z_i z_{i+1} - h_i \sum_i z_i$ ($z_i \in \{\pm 1\}$)

3. Find optimal $\{\gamma^*, \beta^*\}$



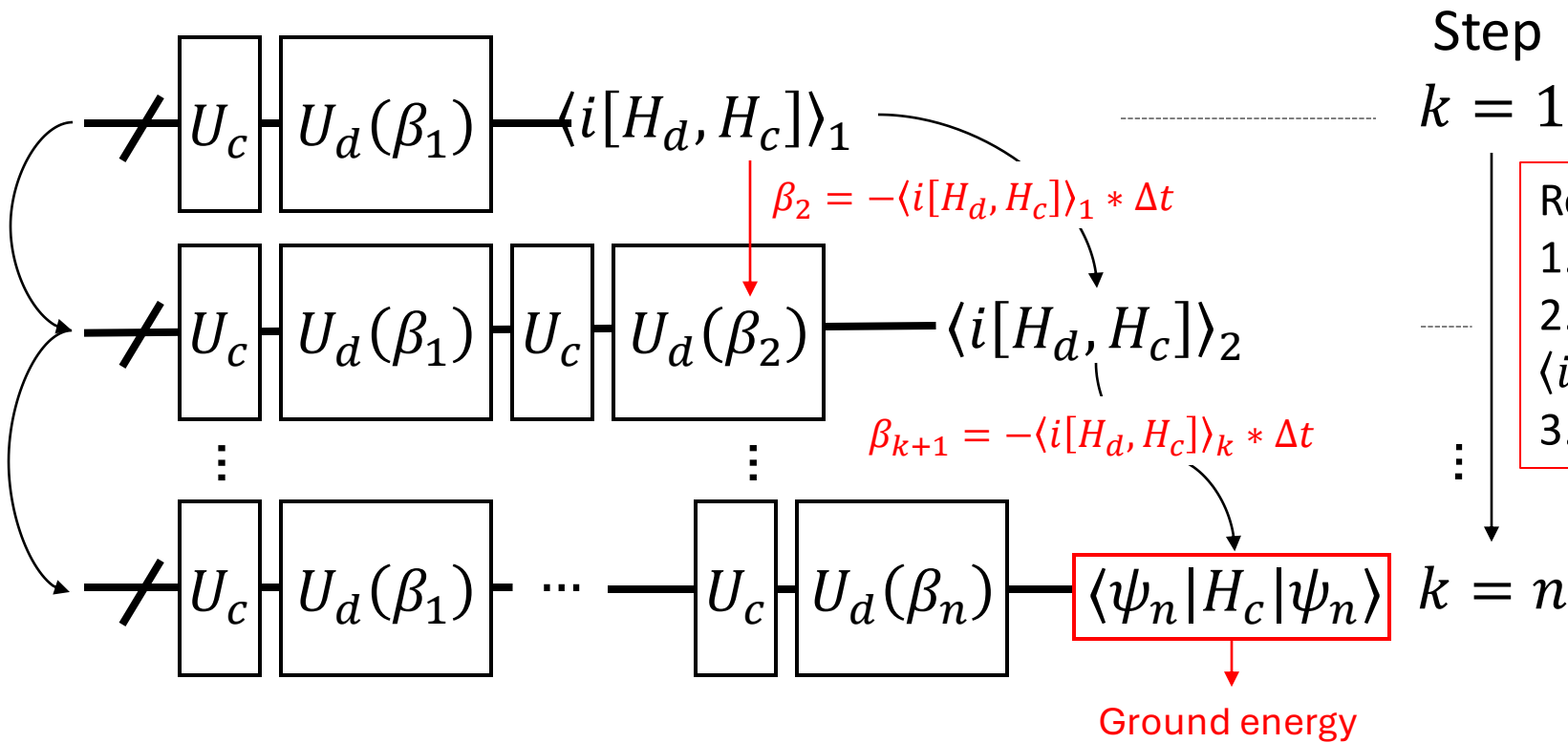
Min/Max cut



Vehicle routing

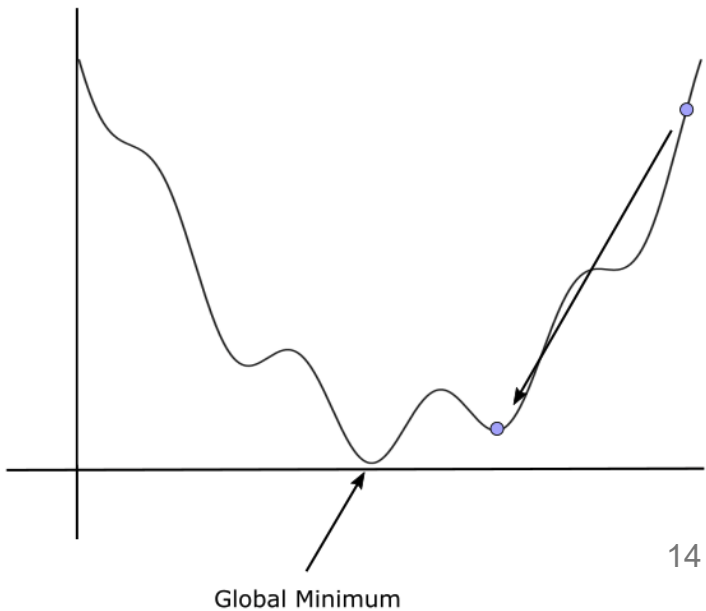
1.6.2. Quantum optimization

QAOA → FALQON $\langle i[H_d, H_c] \rangle_k \equiv \langle \psi_1 | i[H_d, H_c] | \psi_1 \rangle$



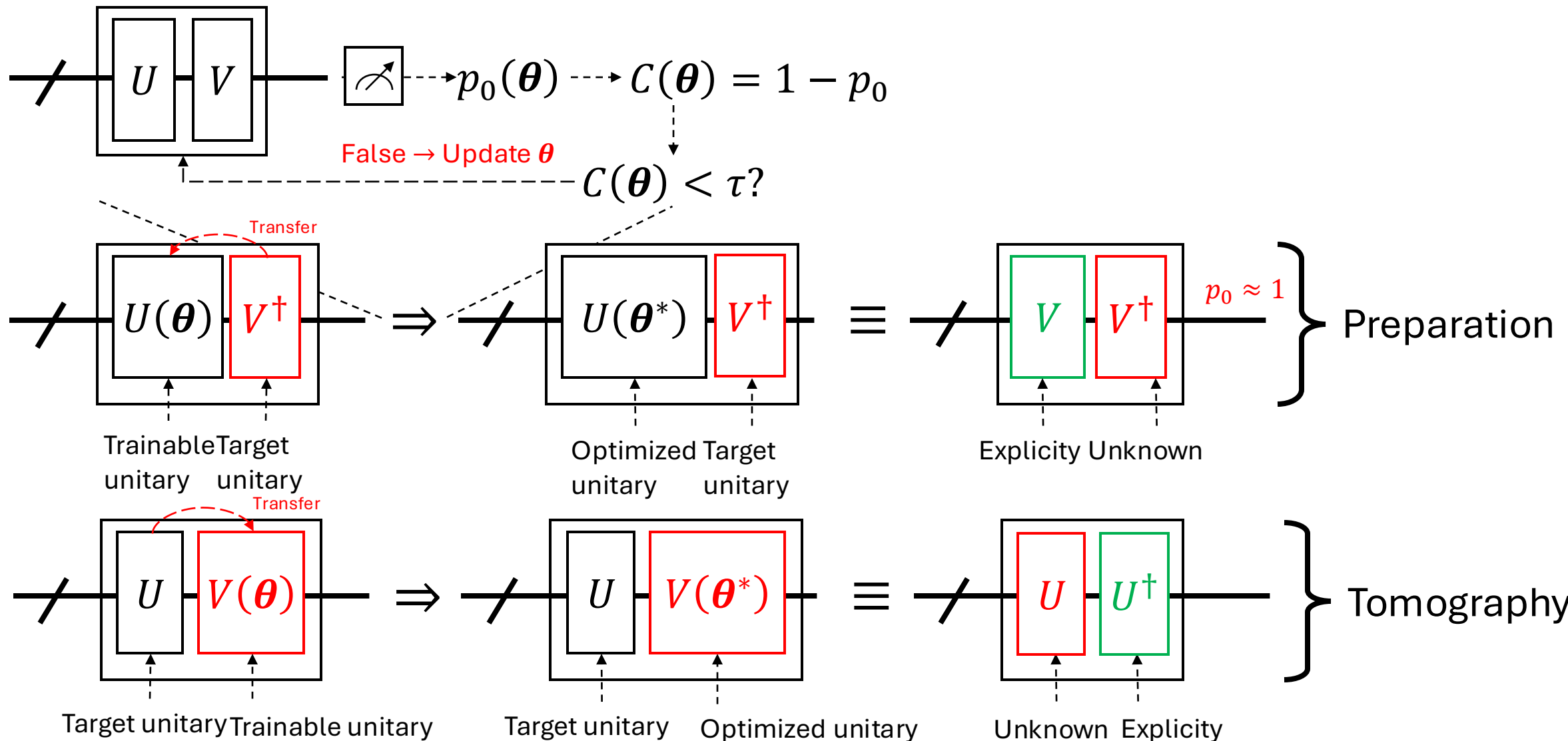
1. Problem Definition Objective Avoid local minimals
2. Hamiltonian $H = H_c + \beta(t)H_d$
3. Find ground energy corresponding to H_c

[1.34] https://pennylane.ai/qml/demos/tutorial_falqon



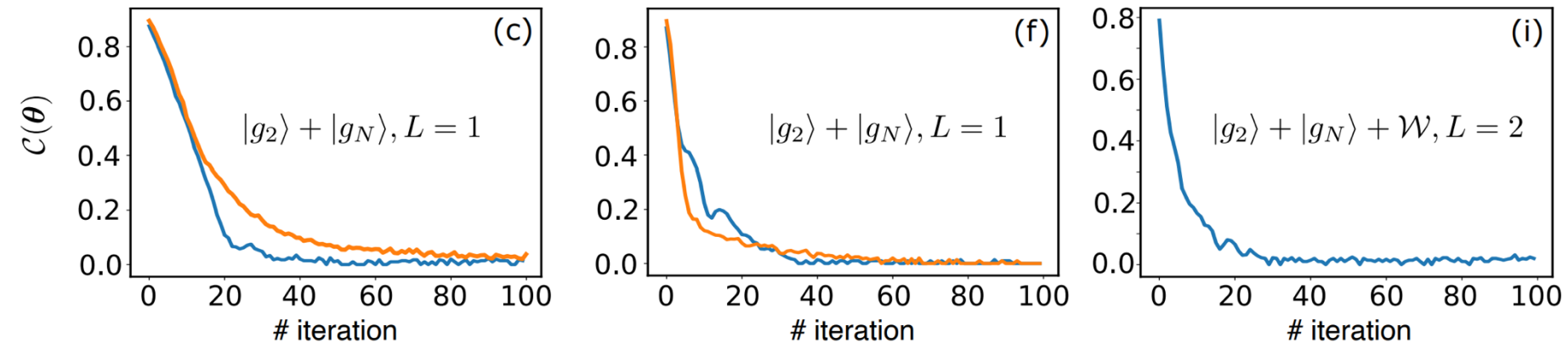
1.6.3. Quantum Optimization

Quantum compiling



1.6.3. Quantum Optimization

Quantum compiling



Some entangled states are hard for preparation:

$$|GHZ_n\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n})$$

$$|W_n\rangle = \frac{1}{\sqrt{n}}(|0\rangle + |1\rangle + \dots + |n\rangle)$$

If

$U(\theta^*) \equiv GHZ^\dagger$

then preparing $|GHZ\rangle$ through $U(\theta^*)$ is better

Almost unitaries acts as “black-box”

U

only being extracted via measurement

If

$V(\theta^*) \equiv U$

we know structure of U via $V(\theta^*)$

PreparationTomography

[1.36] Hai, V. T., & Urbaneja, J. (2025, July). Entangled State Preparation via Cluster States on Quantum Computers with $\langle qo|op\rangle$ Software. In *2025 IEEE International Conference on Quantum Software (QSW)* (pp. 116-122). IEEE.

[1.37] Hai, V. T., & Ho, L. B. (2023). Universal compilation for quantum state tomography. *Scientific Reports*, 13(1), 3750.

1.7. Additional informations

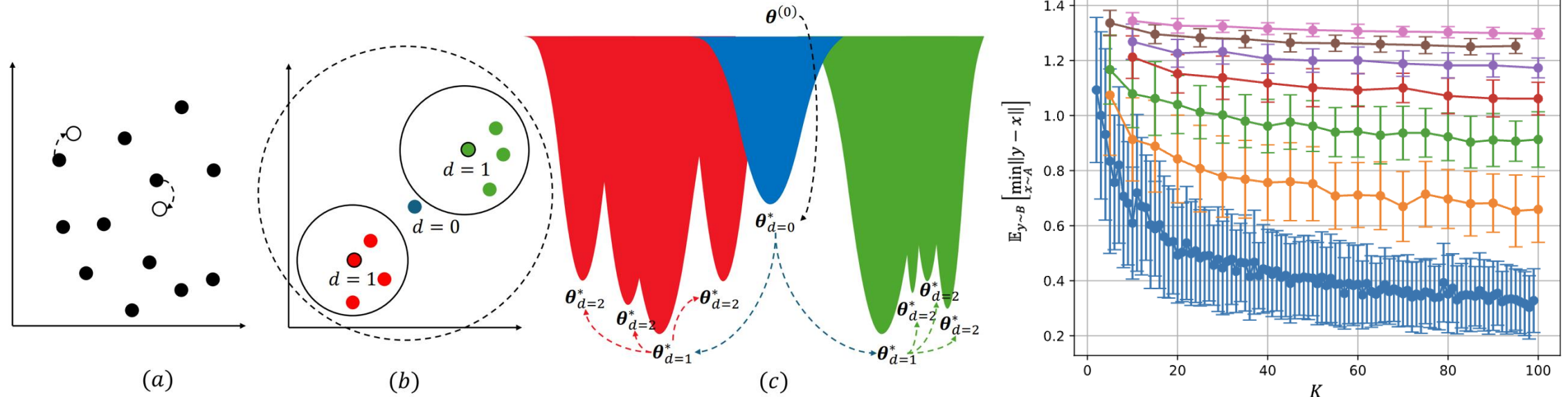
Can we apply classical techniques into quantum optimization?

Consider a set of K optimization targets denoted as $T_1, T_2, \dots, T_K$, all defined over the same search space S . Each target T_k is associated with its own cost function:

$$C(\theta_{(k)}): R^m \rightarrow [0, 1]$$

The goal of multi-target optimization is to find a set of solutions:

$$\{\theta_{(1)}^*, \theta_{(2)}^*, \dots, \theta_{(K)}^*\}$$

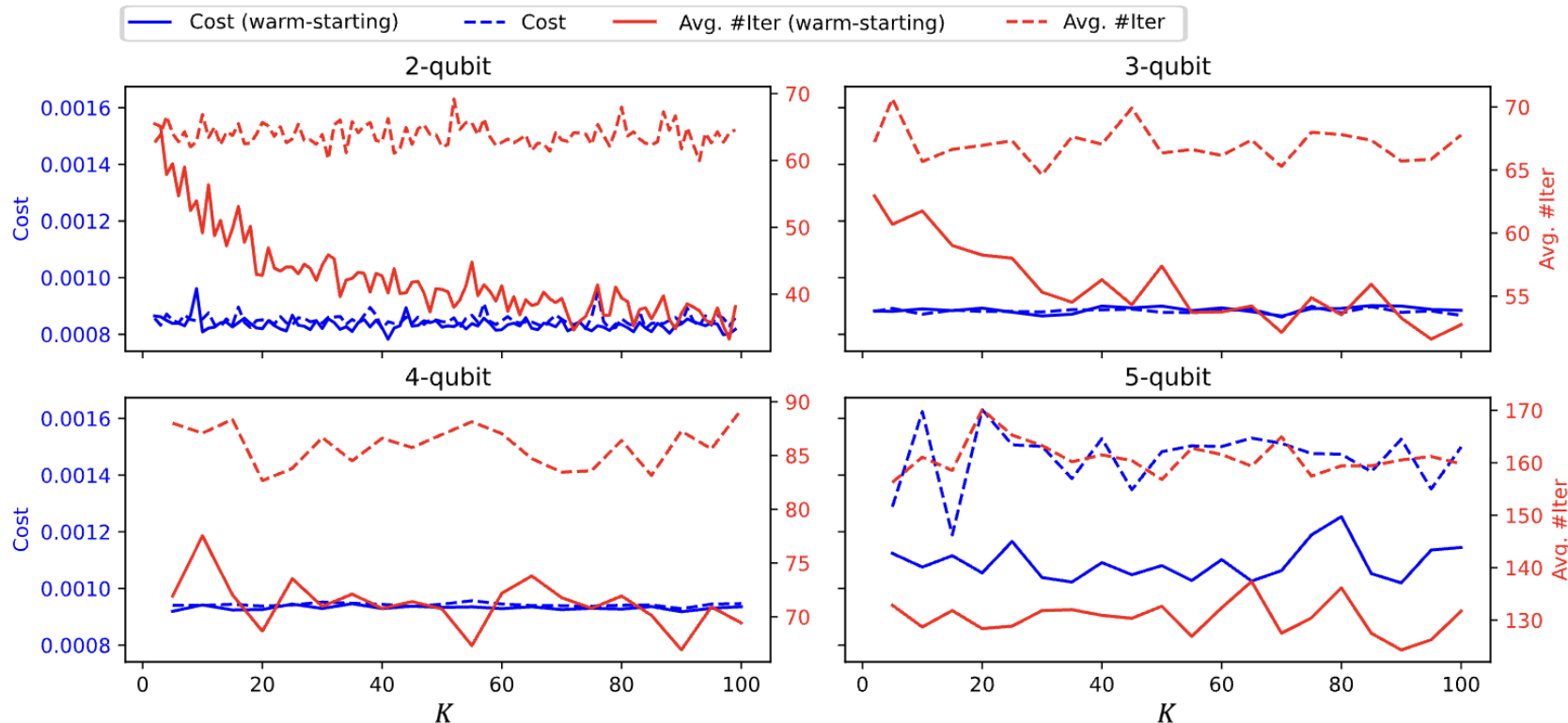


1.7. Additional informations

Can we apply classical techniques into quantum optimization?

Warm-starting method: simply set $\theta_{x(k)}^*$ as initial parameter $\theta_{y(k)}^{(0)} = \theta_{x(k)}^*$.

If $y \approx x + \epsilon$, x and y are targets, considering the first-order Taylor expansion of cost function C with respect to θ at θ_x^* and fixed y , we have:



$$\tilde{\theta}_y^* \approx \theta_x^* - \frac{\nabla_{\theta} C_y(\theta_x^*)}{\|\nabla_{\theta} C_y(\theta_x^*)\|^2} C_y(\theta_x^*)$$

Parameter estimator

2. Quantum machine learning (QML)

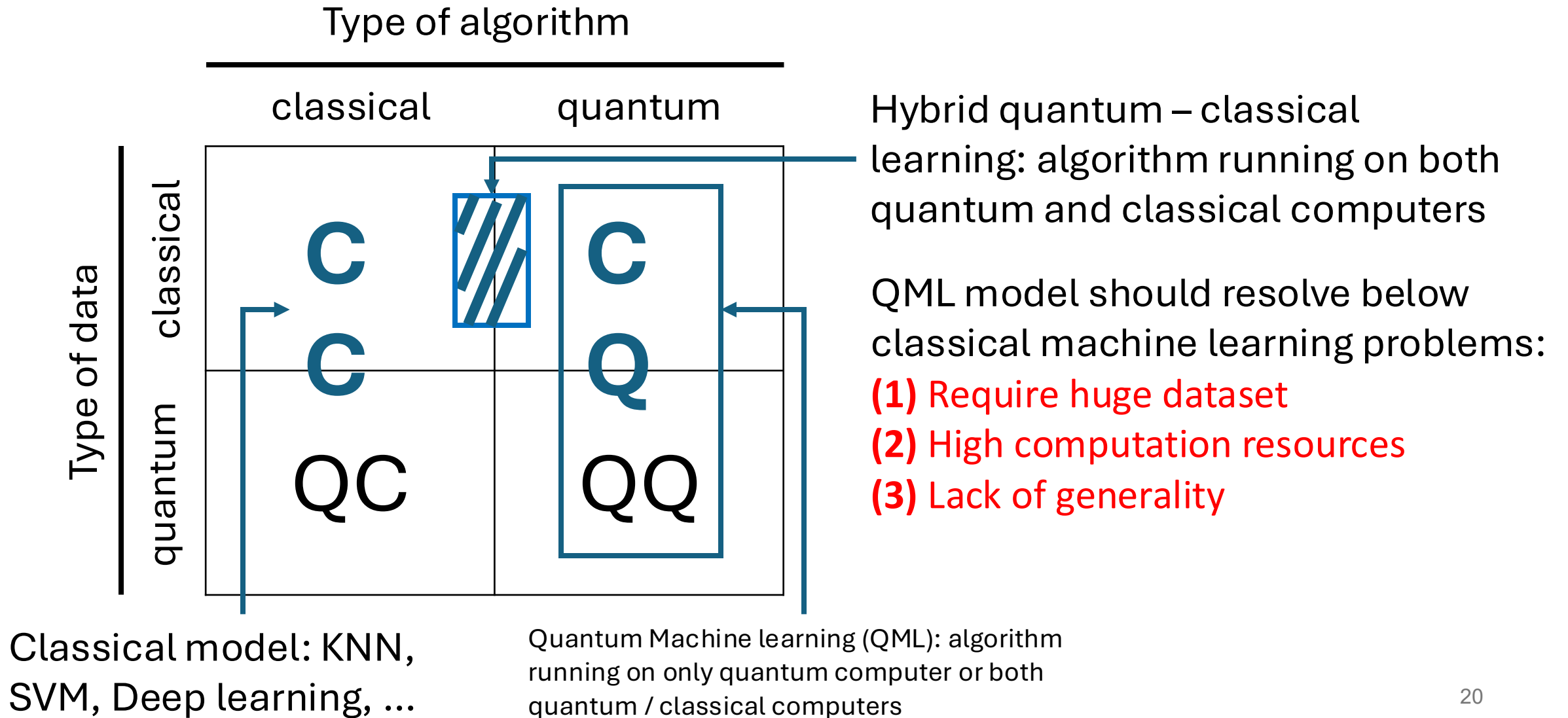
2.1. Overview

2.2. Quantum machine learning algorithms

2.3. Quantum deep learning

2.4. Additional information

2.1. Overview



2.1. Overview

What QML models can do?

Based on quantum advantage **[2.2]**

(1) Require huge dataset

⇒ Require less items

(2) High computation resources

⇒ Lower time complexity

(3) Lack of generality

⇒ Example: Require less #parameter than classical ML model or achieve higher accuracy on the same dataset.

**Be careful with hype
QML papers!**

[2.1] Simões, R. D. M., Huber, P., Meier, N., Smailov, N., Fuchslin, R. M., & Stockinger, K. (2023). Experimental evaluation of quantum machine learning algorithms. *IEEE Access*, 11, 6197-6208.

[2.2] Schuld, M., & Killoran, N. (2022). Is quantum advantage the right goal for quantum machine learning?. *Prx Quantum*, 3(3), 030101.

2.1. Overview

Type of QML model

a) Speed up version of classical ML algorithms:

- QKNN
- QKmean
- QSVM

b) Quantum analogue version of a deep learning model

- QuNN, QCNN
- QGNN
- QGAN

c) Quantum learning

→ I don't know this type

2.1. Overview

Pioneers in QML



Maria Schuld

Xanadu.ai
(PennyLane)



Ryan Babbush

Google Quantum



John Preskill

Caltech

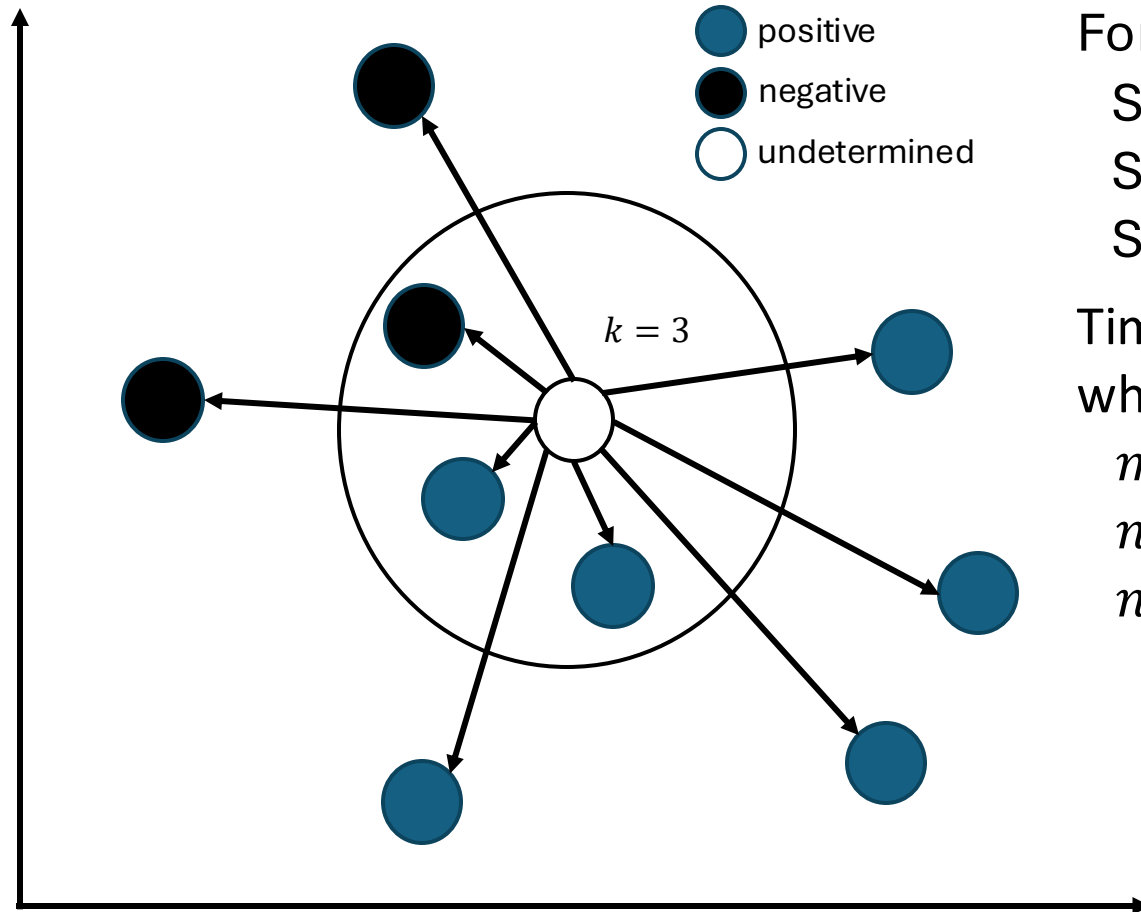


Hsin-Yuan Huang

Google Quantum AI

2.2. QML algorithms

QkNN



For each test data point a :

Step 1: Calculate $Ds_a = \text{dist}(a, x_i) \forall i = 1..n$

Step 2: Sort and get k smallest dist in Ds_a .

Step 3: Get k corresponding label and do major vote

Time complexity: $\mathcal{O}(mn n_{\text{test}})$

where:

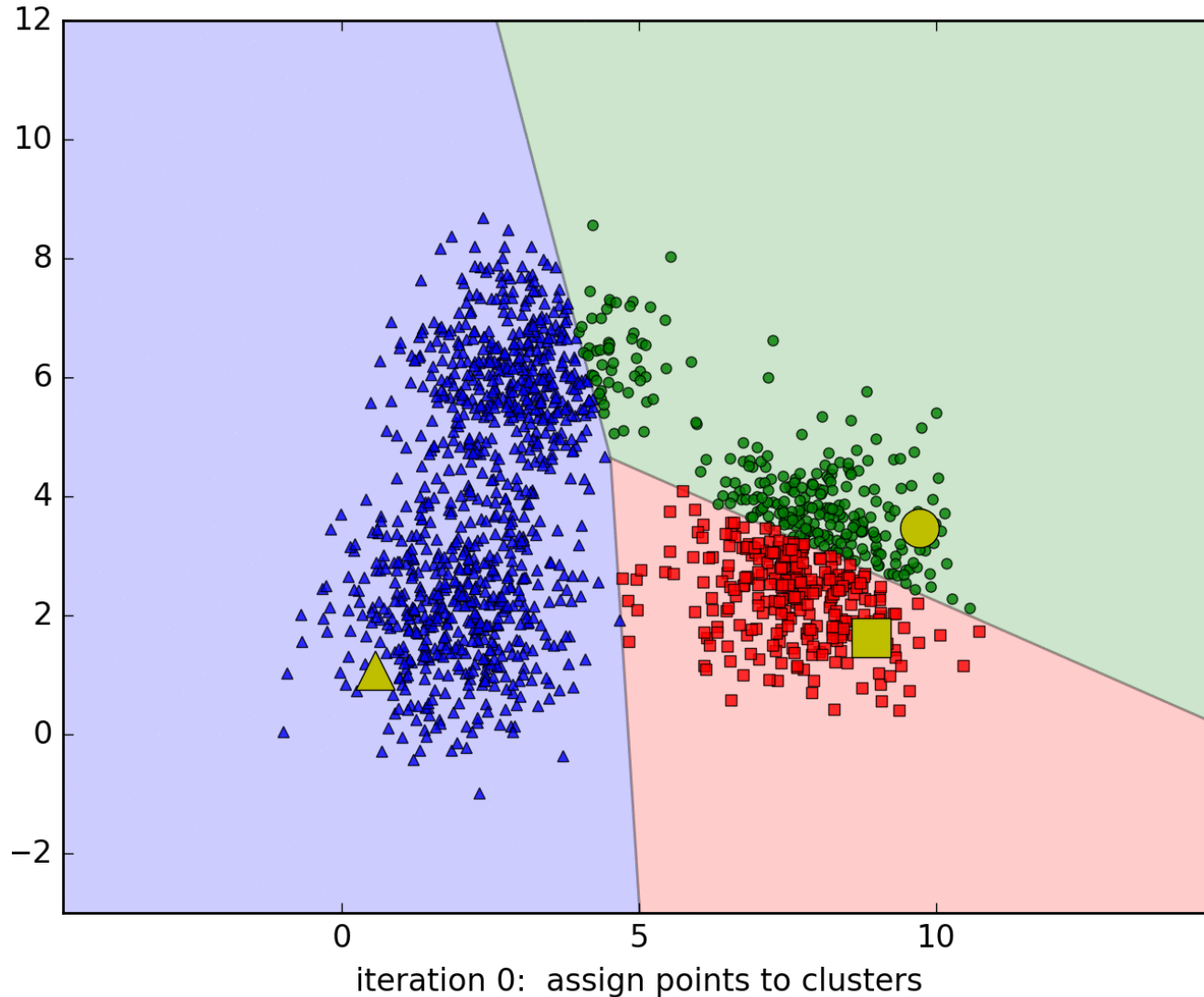
m : dimensional of x_i

n : # train data points

n_{test} : # test data points

2.2. QML algorithms

k -means



Step 1. Select k random centers.

Step 2. Assign each data point to the closest center.

If the assigned labels do not change, then stop.

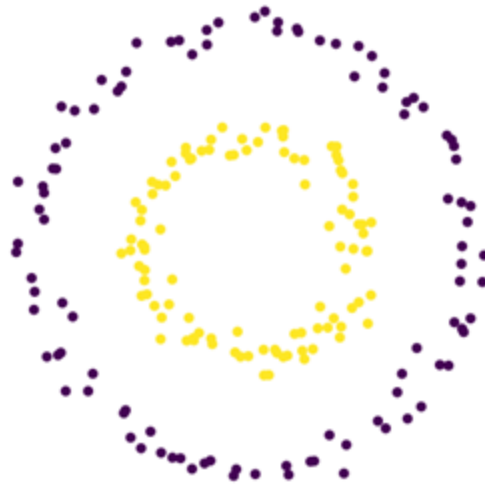
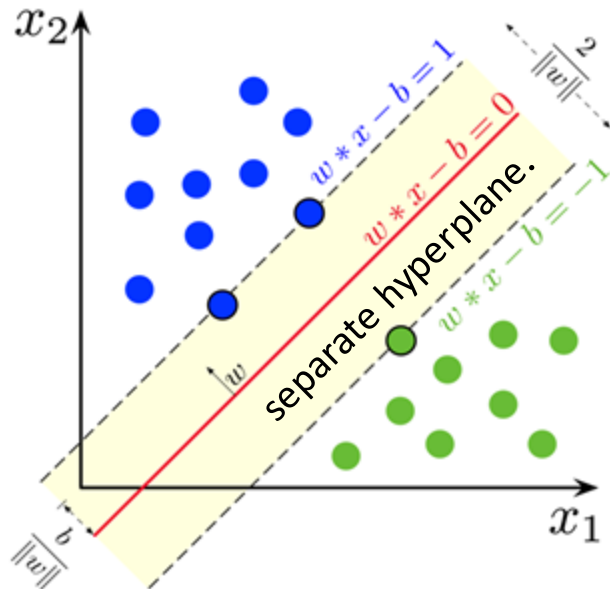
Step 3. New center = average of all assigned points

Step 4. Return to step 2.

Swap-test procedure
can be applied again!

2.2. QML algorithms

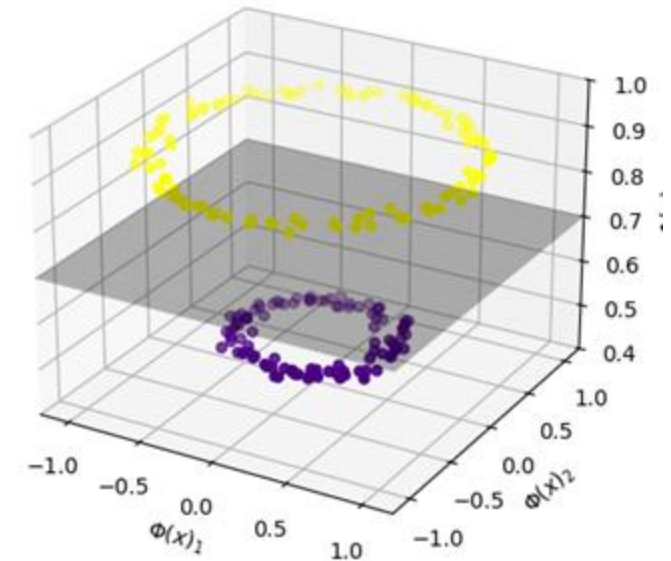
Quantum SVM



Inseparable data

$$x = [x_1 \quad x_2]^T$$

Kernel method



Separable data

$$\phi(x) = [x_1 \quad x_2 \quad x_1 x_2]^T$$

We have: parameters

$$f^*(x) = \text{sgn}(w \cdot x \pm b),$$

with target (w^*, b^*) :

$$\min_{w, b, c} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \epsilon_i$$

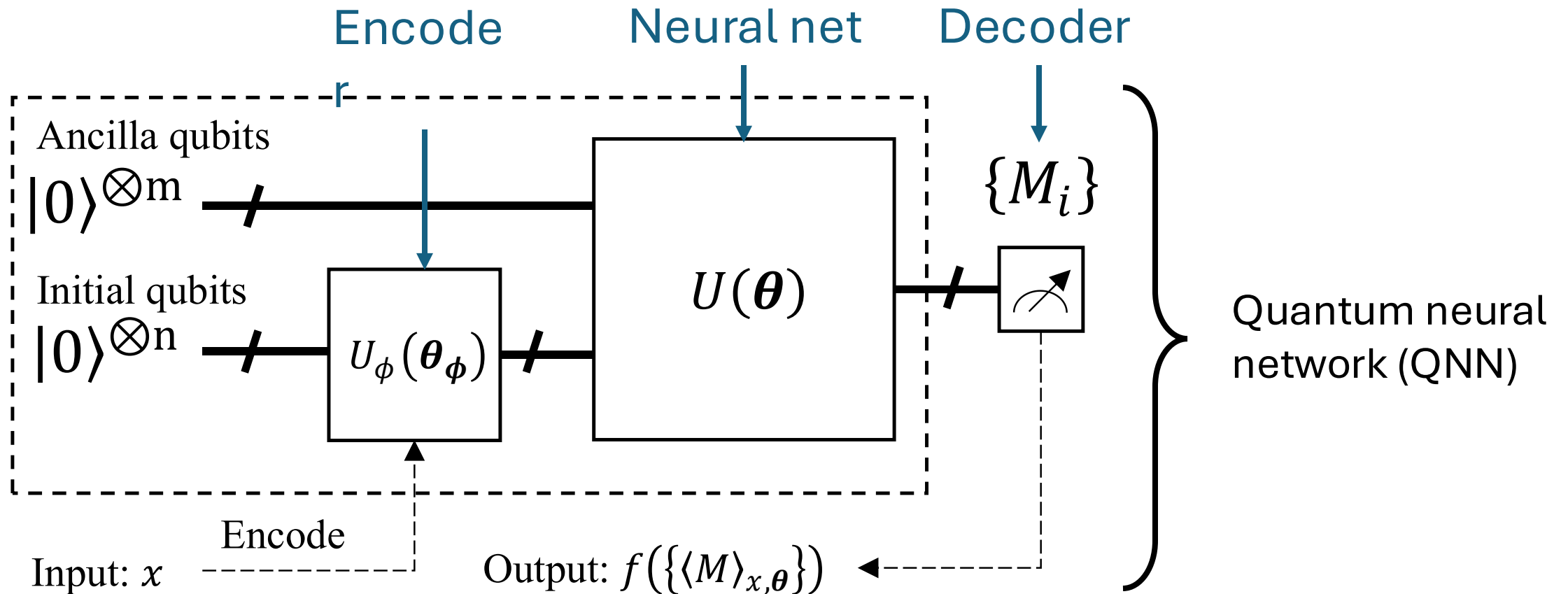
$$\max_{\beta} L(\beta) = \sum_{i=1}^n \beta_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j y_i y_j \phi(x_i)^T \phi(x_j)$$

$$K(x_i, x_j) \equiv \phi(x_i)^T \phi(x_j)$$

Kernel function is faster!

Inner product
 $\mathcal{O}(M^2)$ complex

2.3. Quantum deep learning



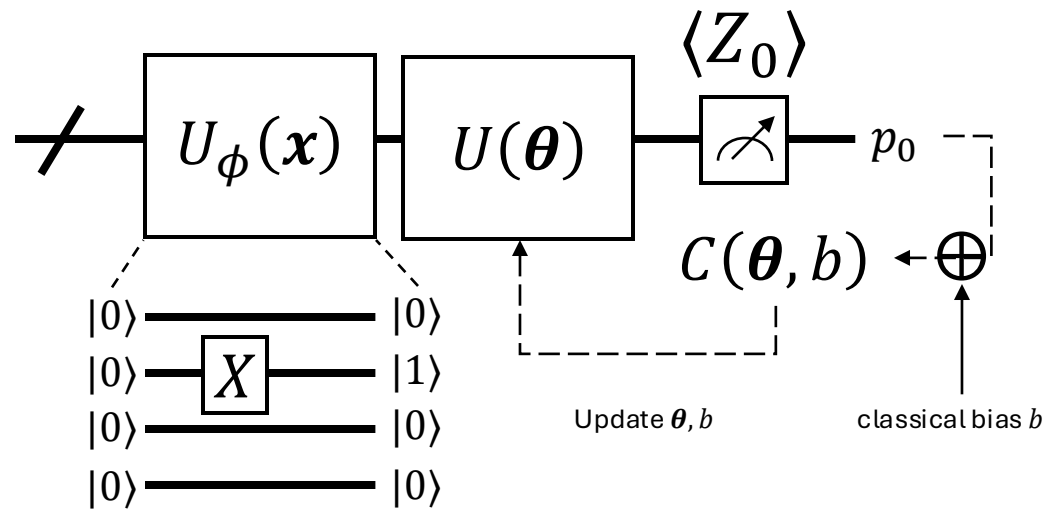
2.3. Quantum deep learning

QNN

Attribute	Classical deep learning	Quantum deep learning
Main concept	Neural network	Parameterized quantum circuit (PQC)
Parameter	Weight $\{w_i\}$ in nodes	Angle value $\{\theta_i\}$ in rotation gates $\{R_X, R_Y, R_Z\}$
Gradient	Backpropagation	General parameter-shift rule and gradient tensor metric
Optimizer	Classical optimizer	Classical optimizer + gradient-free optimizer
Loss	Various type	Associated with expectation values

2.3. Quantum deep learning

Variational classifier for simple classification problem



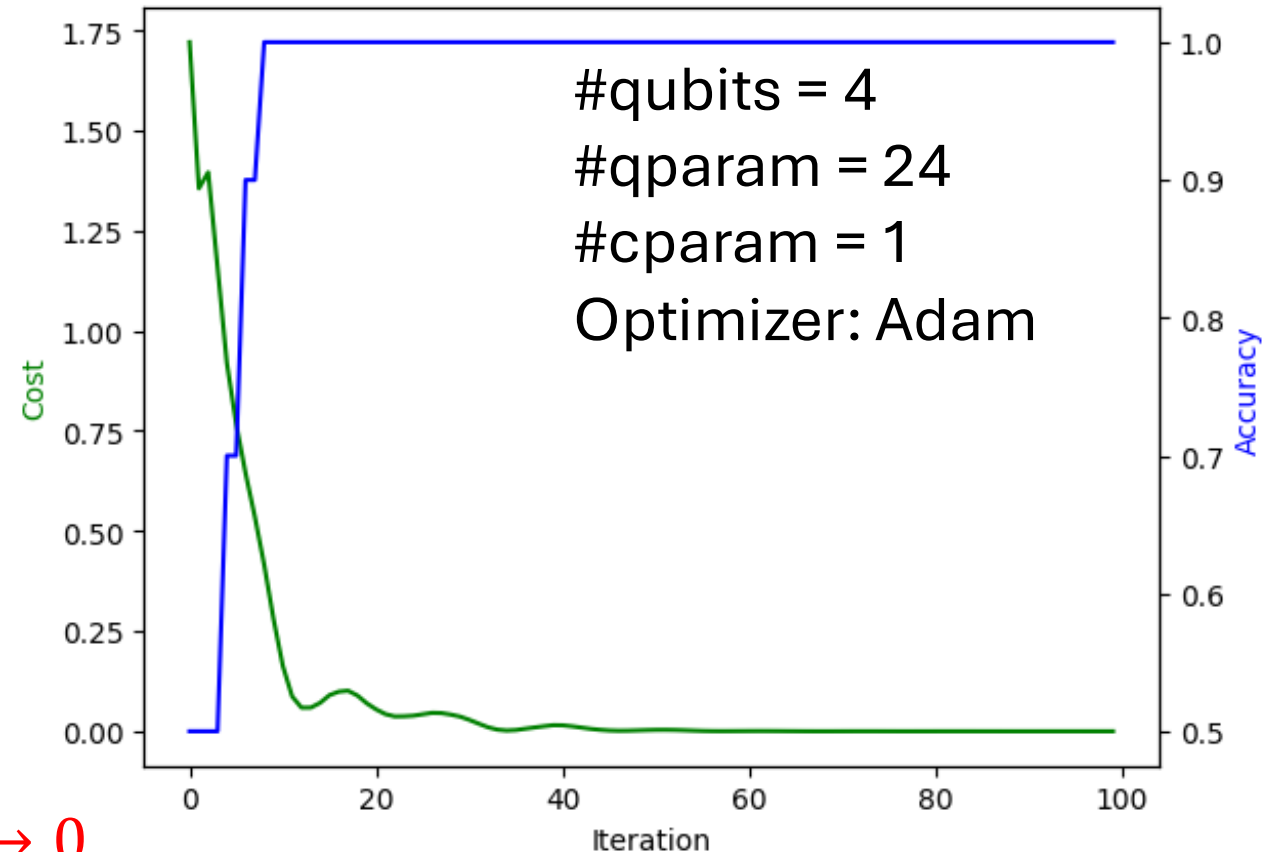
Problem:

Simulate $f: x \in \{0,1\}^{\otimes n} \rightarrow y = \bigoplus_i x_i$

Example: $f(|0100\rangle) = 1$

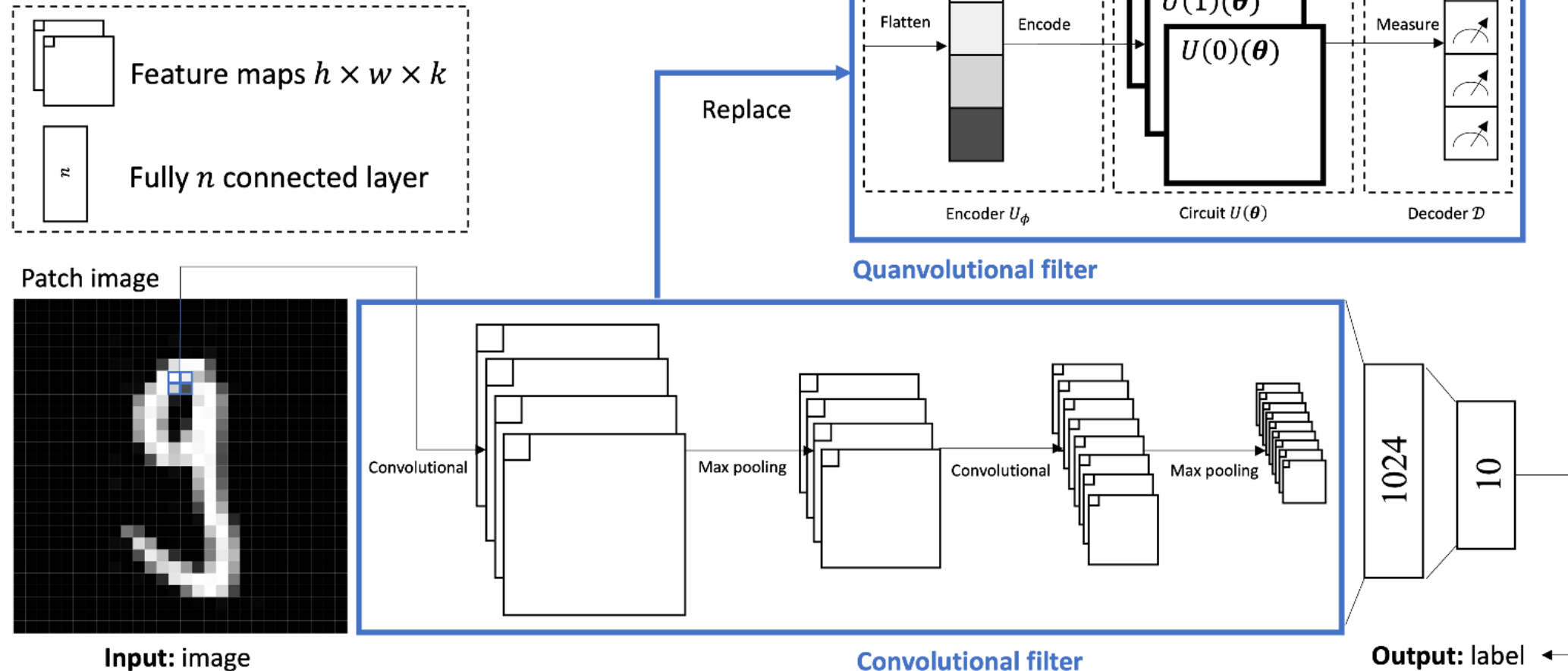
Cost function $C(\theta, b) = \frac{1}{N} \sum (\hat{y}_i - y_i)^2 \rightarrow 0$

Accuracy metric $A = \frac{1}{N} \sum \mathbb{I}(|y_i - \hat{y}_i| < \tau) \rightarrow$



2.3.1. Quantum deep learning

Quantum convolutional NN (QuNN)

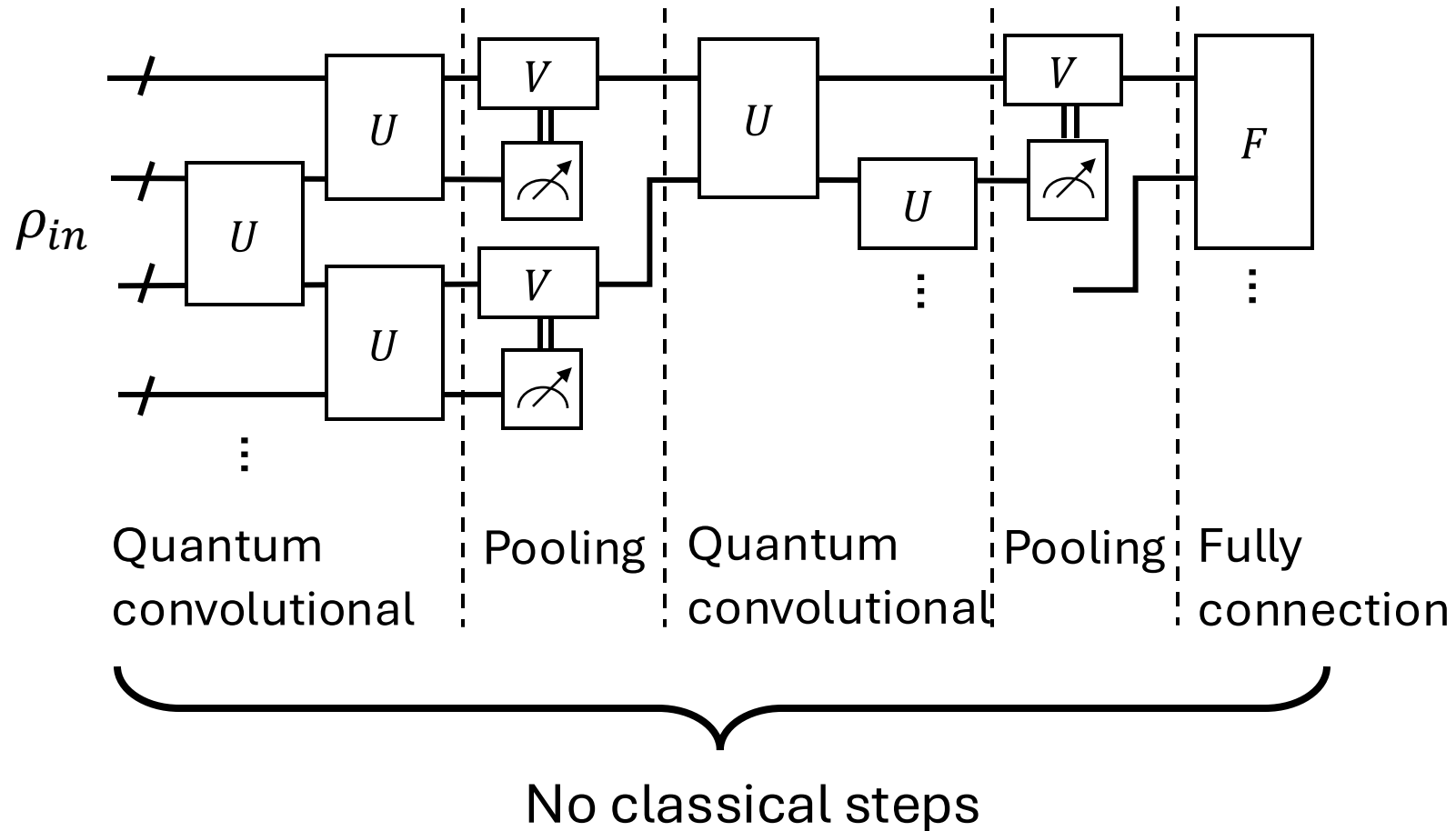


[2.6] Y. Lecun et al, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86(11) (1998) 2278–2324

[2.7] T. Hur et al, Quantum convolutional neural network for classical data classification, Quantum Machine Intelligence 4 (Feb 2022) p. 3.

2.3.2. Quantum deep learning

Quantum Convolution NN (QCNNN)



2.4. Additional information

Power of QML

We will focus on PQC which is the main computation in QML

Encoder: use a very small qubit ($\log_2 N$) to loading / encoding N – dimensional normalized x .

Operator: like classical NN, the structure of $U(\theta)$ decides the model extract feature / process information good or bad.

Decoder: quantum information $\rightarrow$ classical information.

2.4. Additional information

Challenge in QML field

- Standard architecture?
- Trainability, Expressibility
- Lack of experiment resources (not enough simulated #qubit to transform toy dataset → real dataset)
- **Hype in QML**: some applied QML models are actually worse than classical models.
- Other problems

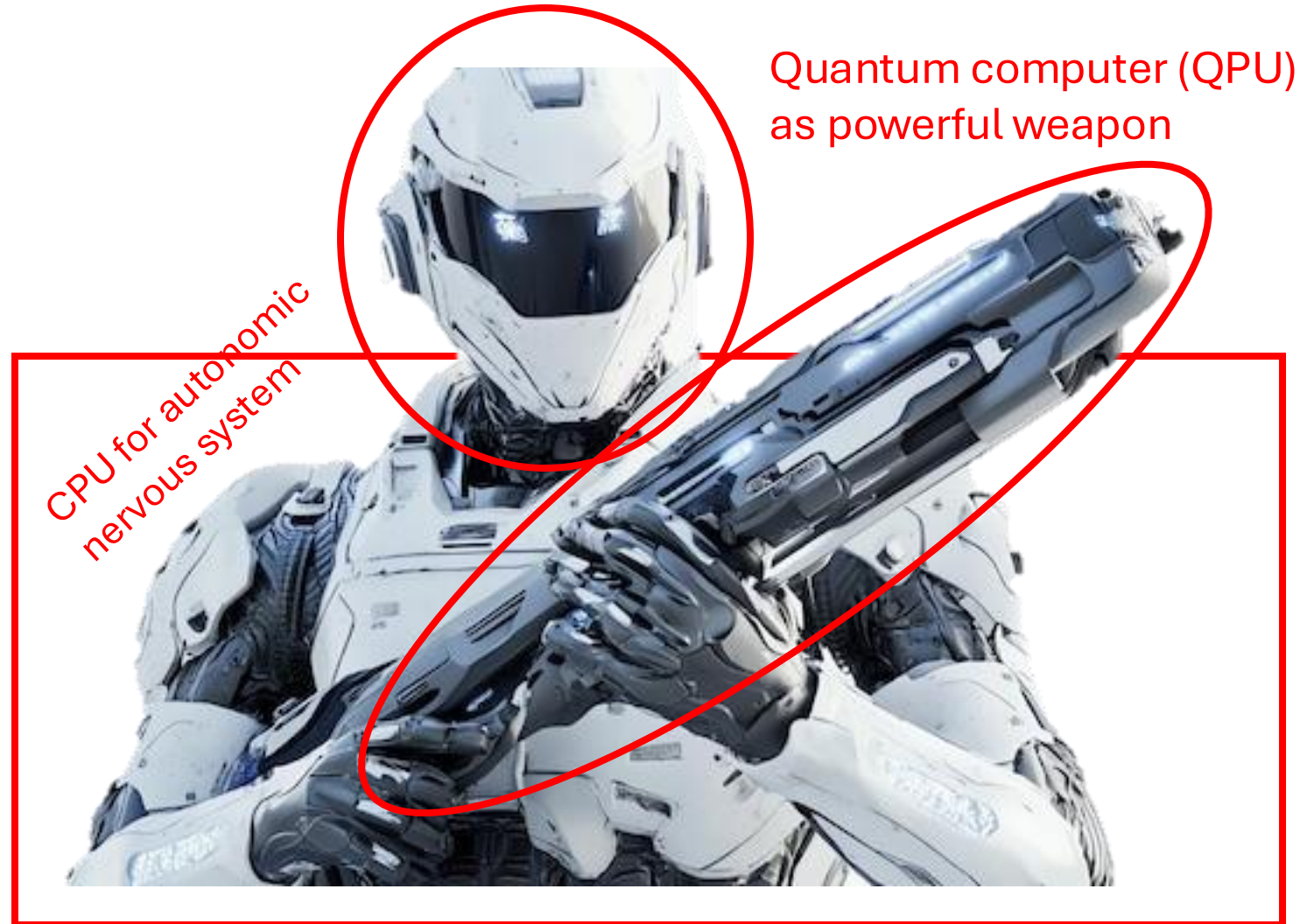
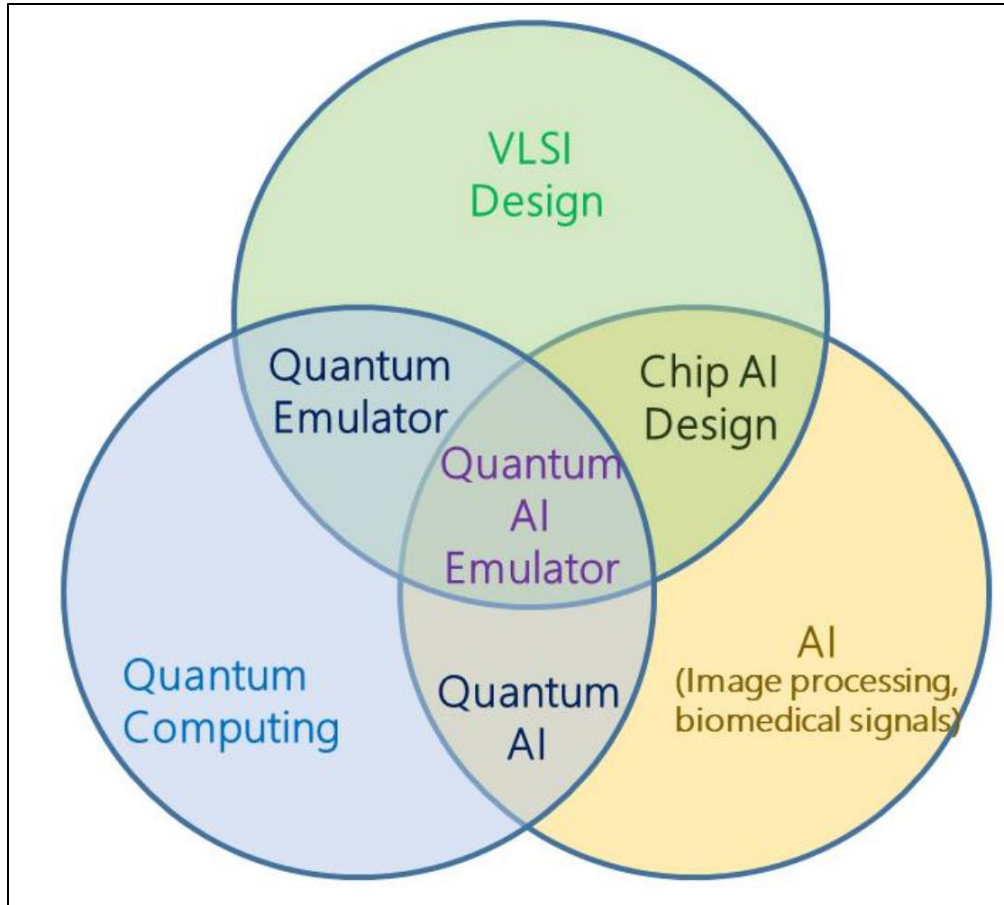
[2.12] Joseph Bowles, Shahnawaz Ahmed, & Maria Schuld. (2024). Better than classical? The subtle art of benchmarking quantum machine learning models.

[2.13] Schuld, M., & Killoran, N. (2022). Is Quantum Advantage the Right Goal for Quantum Machine Learning?. PRX Quantum, 3, 030101.

[2.14] Jens Eisert, & John Preskill. (2025). Mind the gaps: The fraught road to quantum advantage.

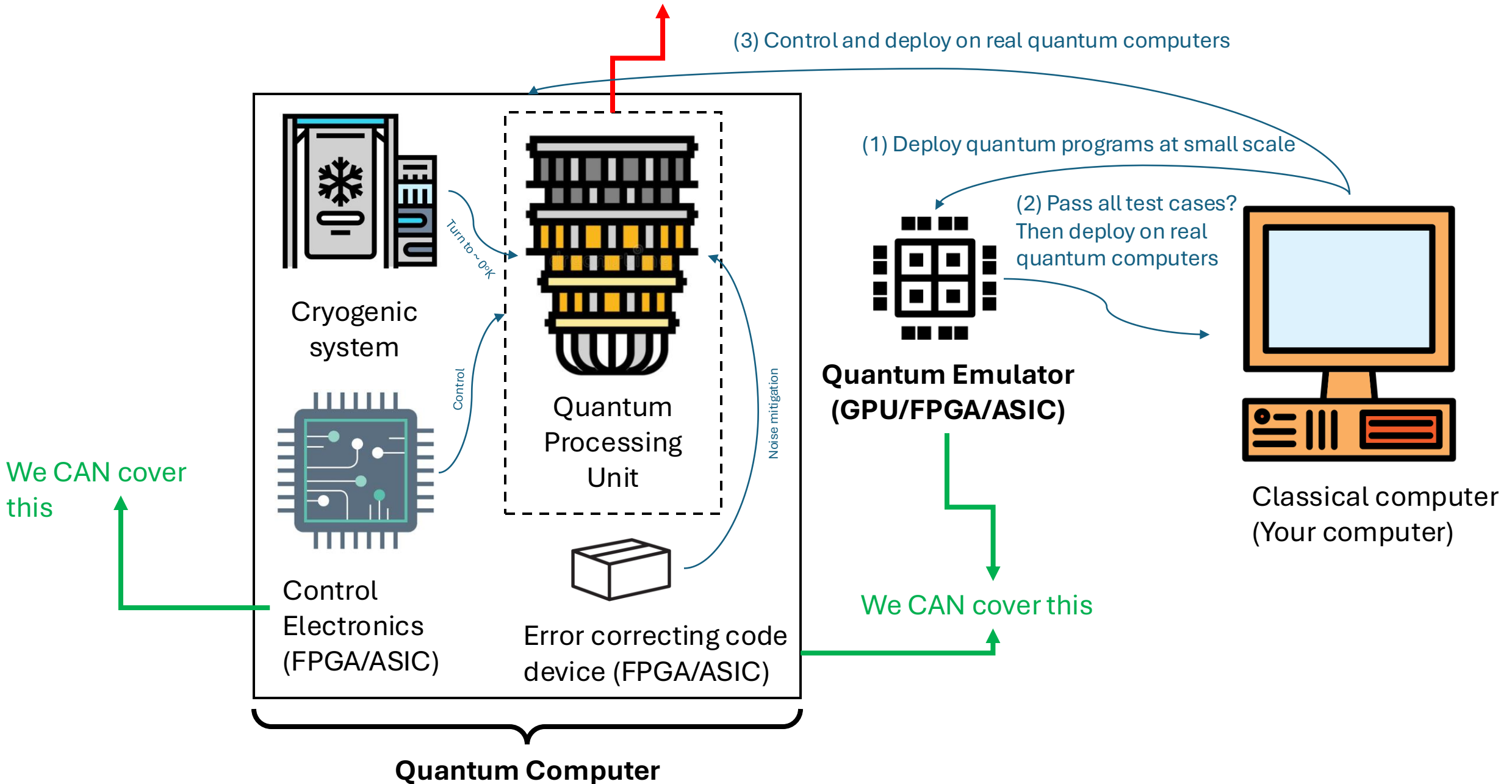
Accelerators for quantum computer: interdisciplinary research direction between “Vi mạch” & Quantum

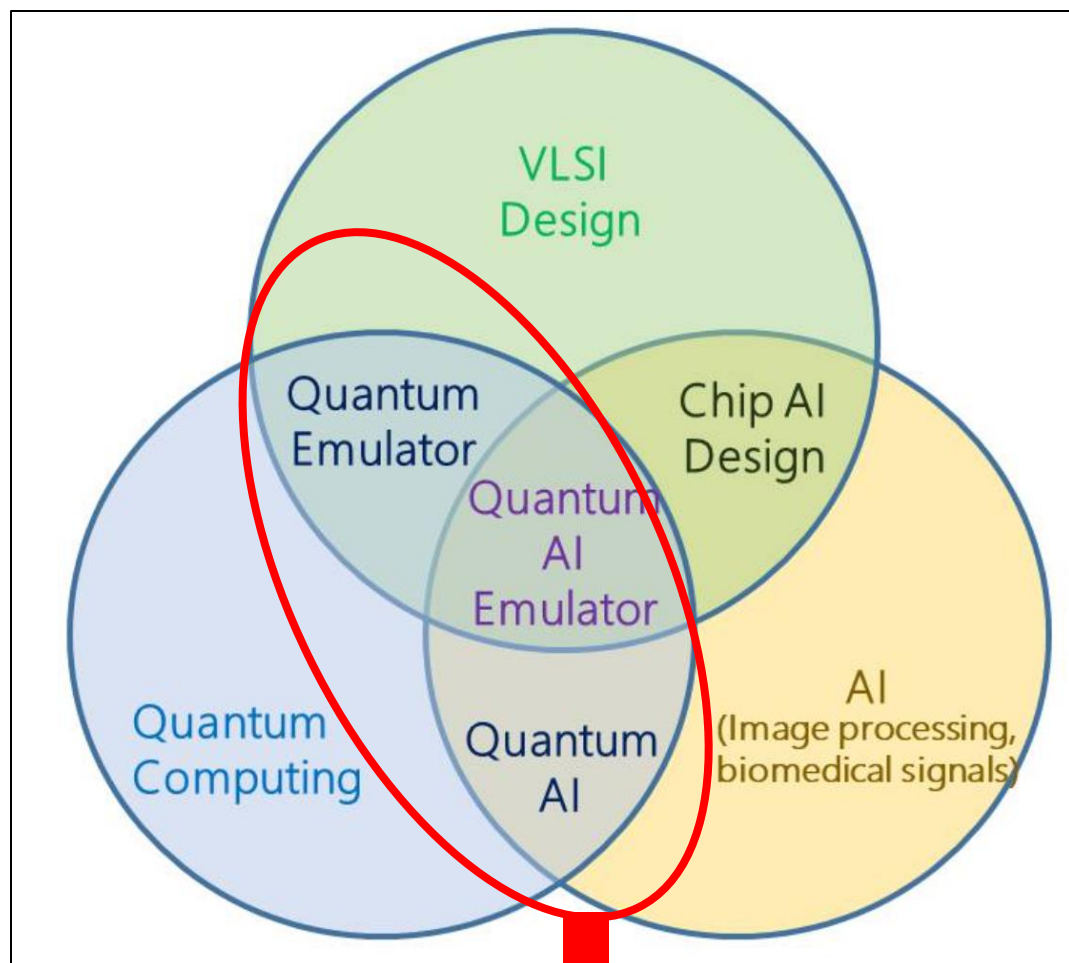
AI (GPU, TPU, ...) for 'smart' control



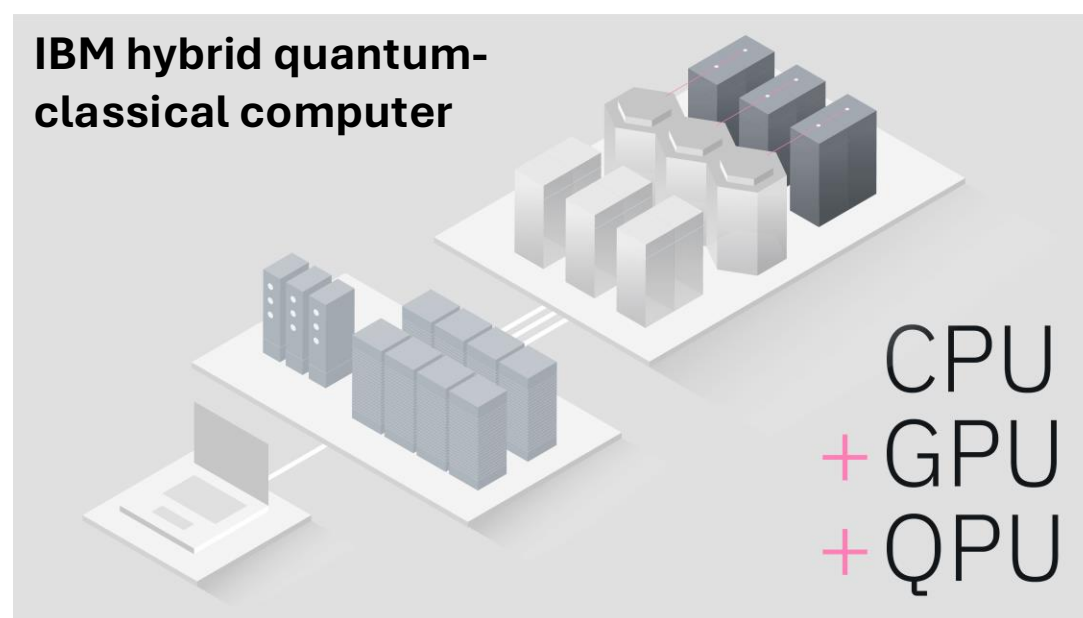
VLSI design as a backbone for whole system

Heart of quantum computers,
we **can not** cover this part!

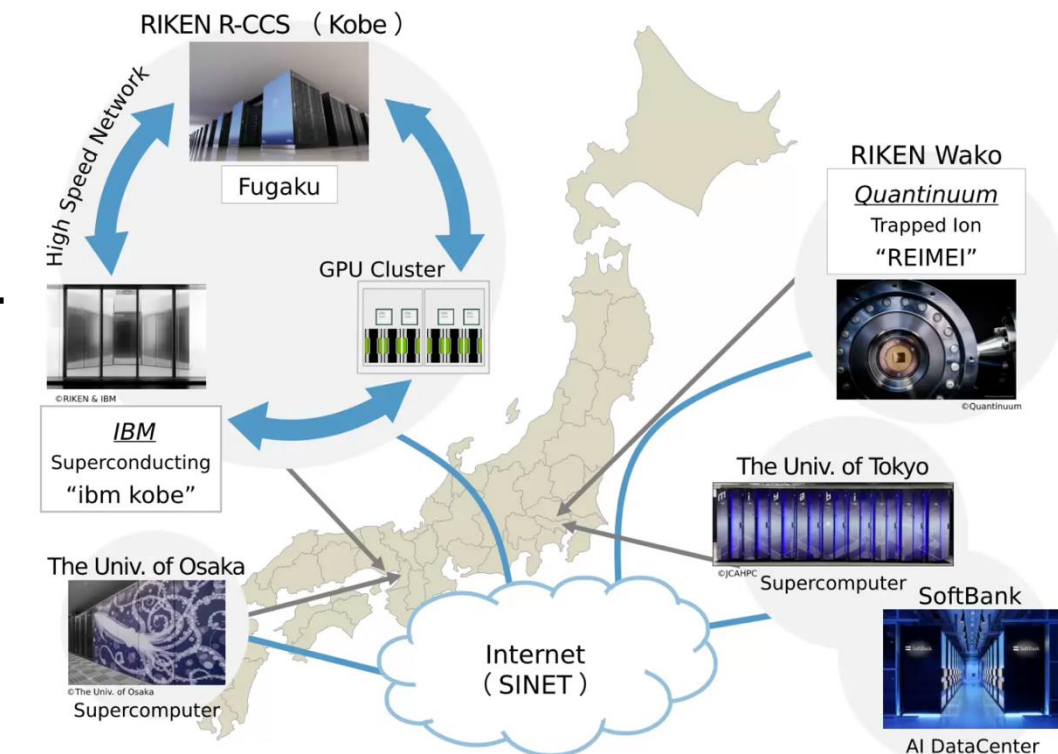




USA



HPCi Japan



Credit: JHPC-quantum

1.1. Quantum simulation approach

Table 1.1. My research’s contribution.

Note: △: Not implemented ○: Implemented/Available ⊙: Improved

Approach	Device	Type	Theory analysis	Software design	Hardware design
Density matrix variant (state vector) [2]	FPGA ZCU102	Emulator	⊙	○	⊙
Wave function variant (state vector) [3]	FPGA ZCU102	Emulator	⊙	○	⊙
Extended stabilizer formalism [4]	GPU RTX4090	Simulator	⊙	⊙	△
Tensor-network [5]	All related works are only focus on one of these aspects!				
Decision diagram [6]					

[2] Ali Javadi-Abhari,, et al. Quantum computing with Qiskit. arXiv preprint arXiv:2405.08810, 2024.

[3] Nguyen et al. Qsun: an open-source platform towards practical quantum machine learning applications. Machine Learning: Science and Technology, 3(1):015034, 2022.

[4] Andi Gu, Salvatore F. E. Oliviero, and Lorenzo Leone. Doped stabilizer states in many-body physics and where to find them. Phys. Rev. A, 110:062427, Dec 2024

[5] Feng Pan, Hanfeng Gu, Lvlin Kuang, Bing Liu, and Pan Zhang. Efficient quantum circuit simulation by tensor network methods on modern gpus. ACM Transactions on Quantum Computing, 5(4), November 2024.

[6] Thomas Grurl and et al. Arrays vs. Decision Diagrams: A Case Study on Quantum Circuit Simulators. In 2020 IEEE 50th International Symposium on Multiple-Valued Logic (ISMVL), pages 176–181, 2020.

1.3. Representation of state & gate

A n – qubit quantum state can be in the form:

$$|\psi^{(t)}\rangle = \begin{bmatrix} \alpha_0^{(t)} \\ \vdots \\ \alpha_{N-1}^{(t)} \end{bmatrix} \quad \text{where } N = 2^n \text{ and } \alpha_j^{(t)} \in \mathbb{C} \\ \|\alpha_j^{(t)}\| \leq 1$$

The number of amplitudes grows exponentially based on #Qubits

A quantum gate update amplitudes $\{\alpha_j^{(t)}\}$:

$$g(w, \theta): \{\alpha_j^{(t)}\} \rightarrow \{\alpha_j^{(t+1)}\} \quad (1.1)$$

Traversing on a long vector

where w and θ are wires and parameter (if applicable).

Our target is to compute the evolved (final) state:

$$|\psi^{(m)}\rangle = U(\boldsymbol{\theta})|\psi^{(0)}\rangle \quad (1.2)$$

Both $|\psi^{(t+1)}\rangle$ and $|\psi^{(t)}\rangle$ need to be saved at the same time

under unitary U . U is composed from gates or sub-operators via tensor product/matrix multiplication

2. A wave-function based emulator (FQsun)

The wave-function approach is based on the representation in the last slides.

Challenges:

- (a) **Efficient Memory Management:** able to process large #Qubits
- (b) **Flexible Quantum Gate Configuration:** an adaptable gate configuration system is needed
- (c) **Optimized Task Scheduling:** the emulator requires an optimized scheduling mechanism to ensure continuous processing without latency.
- (d) **Multi-Level Precision Support:** supporting various precision levels allows the emulator to balance accuracy with resource usage

2.2. Algorithm

Table 2.2. Comparison between variants

(†) $\hat{n}_j \in [1, 2]$

Matrix-vector multiplication

Op	Wave-function	Density-matrix
Mul/gate	$2^{n+\hat{n}_j+1}$ (†)	2^{2n}
Add/gate	$2^{n+\hat{n}_j+1}$ (†)	2^{2n-2}

The final state is calculated through series of $\mathcal{W}$:

$$|\psi^{(m)}\rangle = \mathcal{W}(g_m) \dots \mathcal{W}(g_2)\mathcal{W}(g_1)|\psi^{(0)}\rangle \quad (2.1)$$

The transition function $\mathcal{W}(g_j)$ updates amplitudes $\{\alpha_j\}$ on the $(w_0)^{th}$ qubit:

$$\mathcal{W}(g): \begin{bmatrix} \alpha_{s_i}^{(t)} \\ \alpha_{s_i+2^w}^{(t)} \end{bmatrix} \rightarrow g \begin{bmatrix} \alpha_{s_i}^{(t)} \\ \alpha_{s_i+2^w}^{(t)} \end{bmatrix} = \begin{bmatrix} \alpha_{s_i}^{(t+1)} \\ \alpha_{s_i+2^w}^{(t+1)} \end{bmatrix} \quad (2.2)$$

Algorithm 1 $\mathcal{W}(\{\alpha_j^{(t)}\}, g, \dots)$: Operation of gates from amplitude $head^{th}$ to amplitude $tail^{th}$ on n -qubit state.

Require: $\{\alpha_j^{(t)}\}$, target qubit $w_0 \in [0, n)$, $g = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathcal{G}$ or control-target

qubit $w_0, w_1 \in [0, n), w_0 \neq w_1$, the head ($head$) and tail ($tail$) index of input amplitude.

Ensure: $\sum_j |\alpha_j^{(t)}|^2 = 1$

state $\leftarrow [\text{bin}(0, n), \text{bin}(1, n), \dots, \text{bin}(N-1, n)]$ $\triangleright \text{bin}(i, k)$ is the function that represent i by k binary bit,

for $j \leftarrow [head, head+1, \dots, tail]$ **do**

[Single qubit gate]

cut $\leftarrow 2^{n-w_0-1}$

if state[j][w_0] == 0 **then**

$\alpha_j^{(t+1)} \leftarrow \alpha_j^{(t+1)} + a \times \alpha_j^{(t)}$

$\alpha_{j+cut}^{(t+1)} \leftarrow \alpha_{j+cut}^{(t+1)} + b \times \alpha_j^{(t)}$

else

$\alpha_j^{(t+1)} \leftarrow \alpha_j^{(t+1)} + d \times \alpha_j^{(t)}$

$\alpha_{j-cut}^{(t+1)} \leftarrow \alpha_{j-cut}^{(t+1)} + c \times \alpha_j^{(t)}$

return $\{\alpha_j^{(t+1)}\}$

[2-qubit gate]

cut $\leftarrow 2^{n-w_1-1}$

if state[j][w_0] == 1 **then**

if state[j][w_1] == 1 **then**

$\alpha_{j-cut}^{(t+1)} \leftarrow \alpha_{j-cut}^{(t+1)} + \alpha_j^{(t)}$

else

$\alpha_{j+cut}^{(t+1)} \leftarrow \alpha_{j+cut}^{(t+1)} + \alpha_j^{(t)}$

else

$\alpha_j^{(t+1)} \leftarrow \alpha_j^{(t)}$

Depend on type

Algorithm 2.1. The implementation of single- and multiple-qubit gates is unified into a common framework

2.3. System overview/Ideals

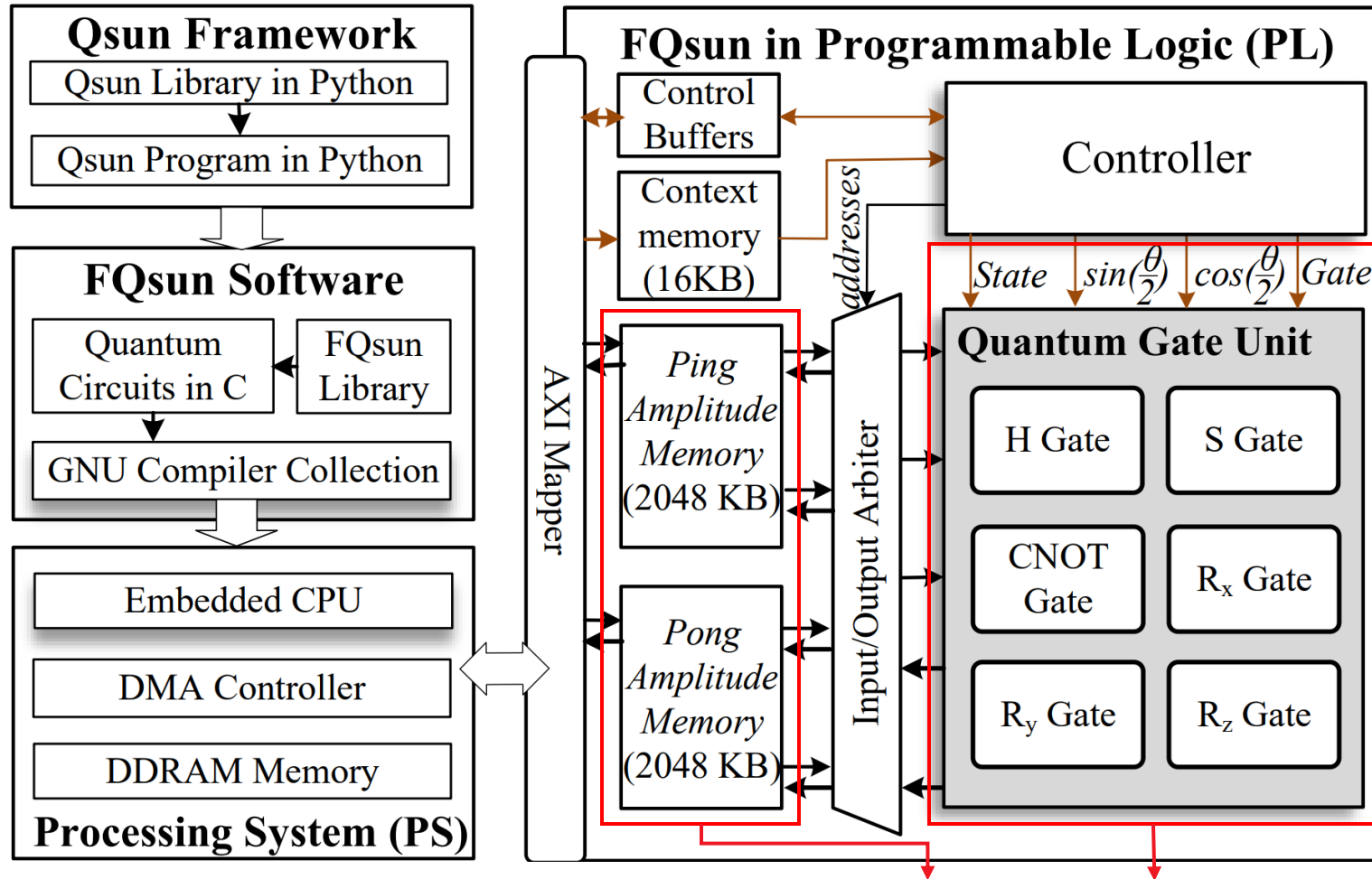


Figure 2.1. Overview architecture of our FQsun at the system-on-chip (SoC) level on FPGA.

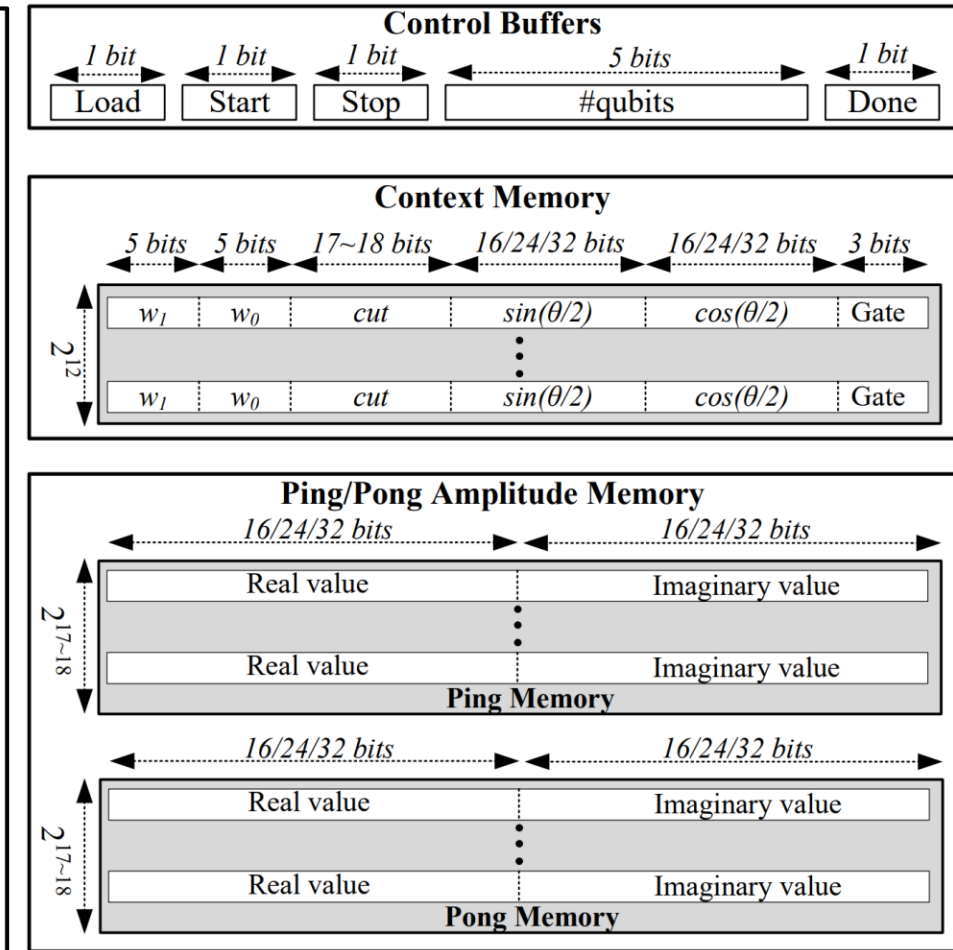
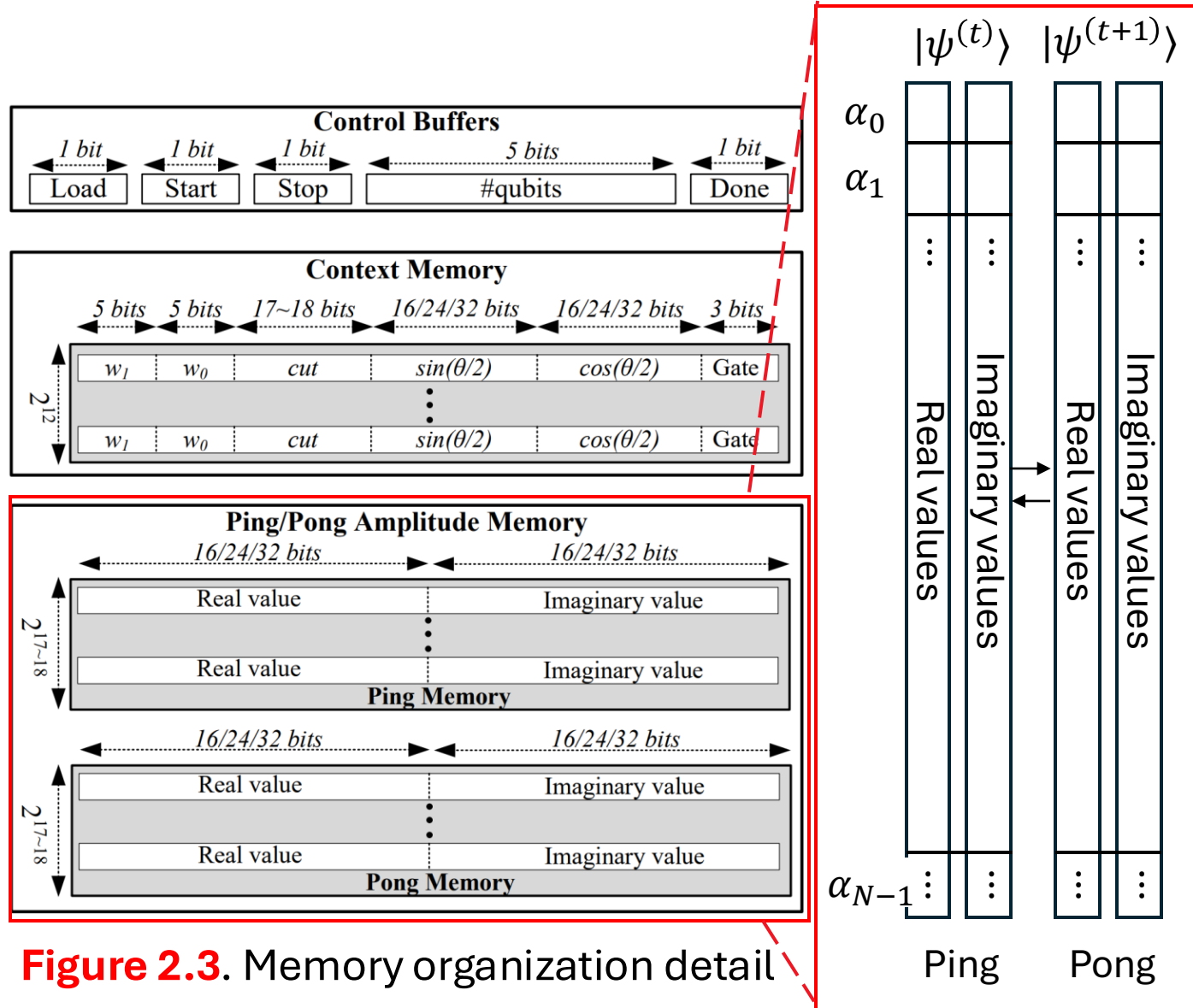


Figure 2.2. Memory organization of the proposed FQsun.

2.3.1-2. **Idea #1:** efficient memory organization



Idea #2: optimized scheduling

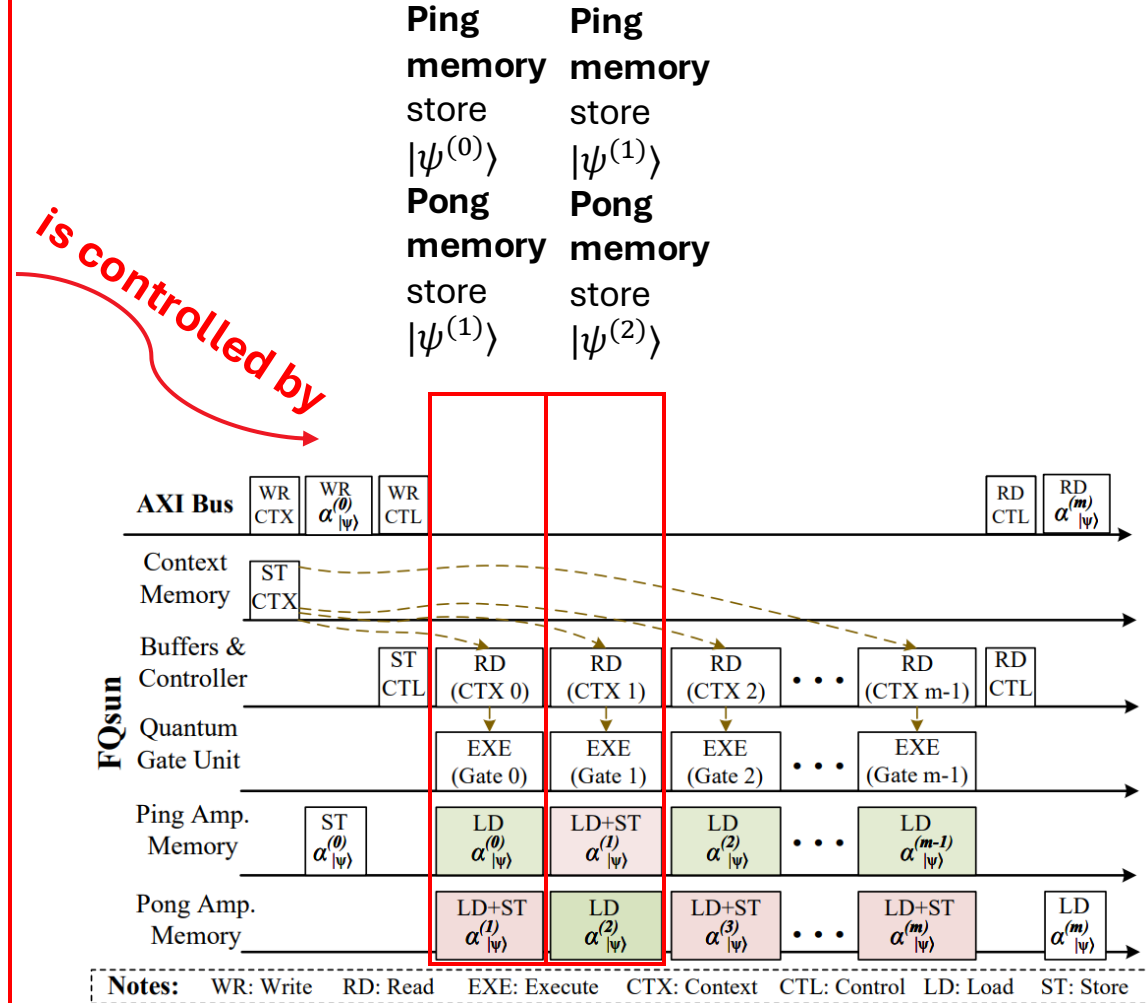
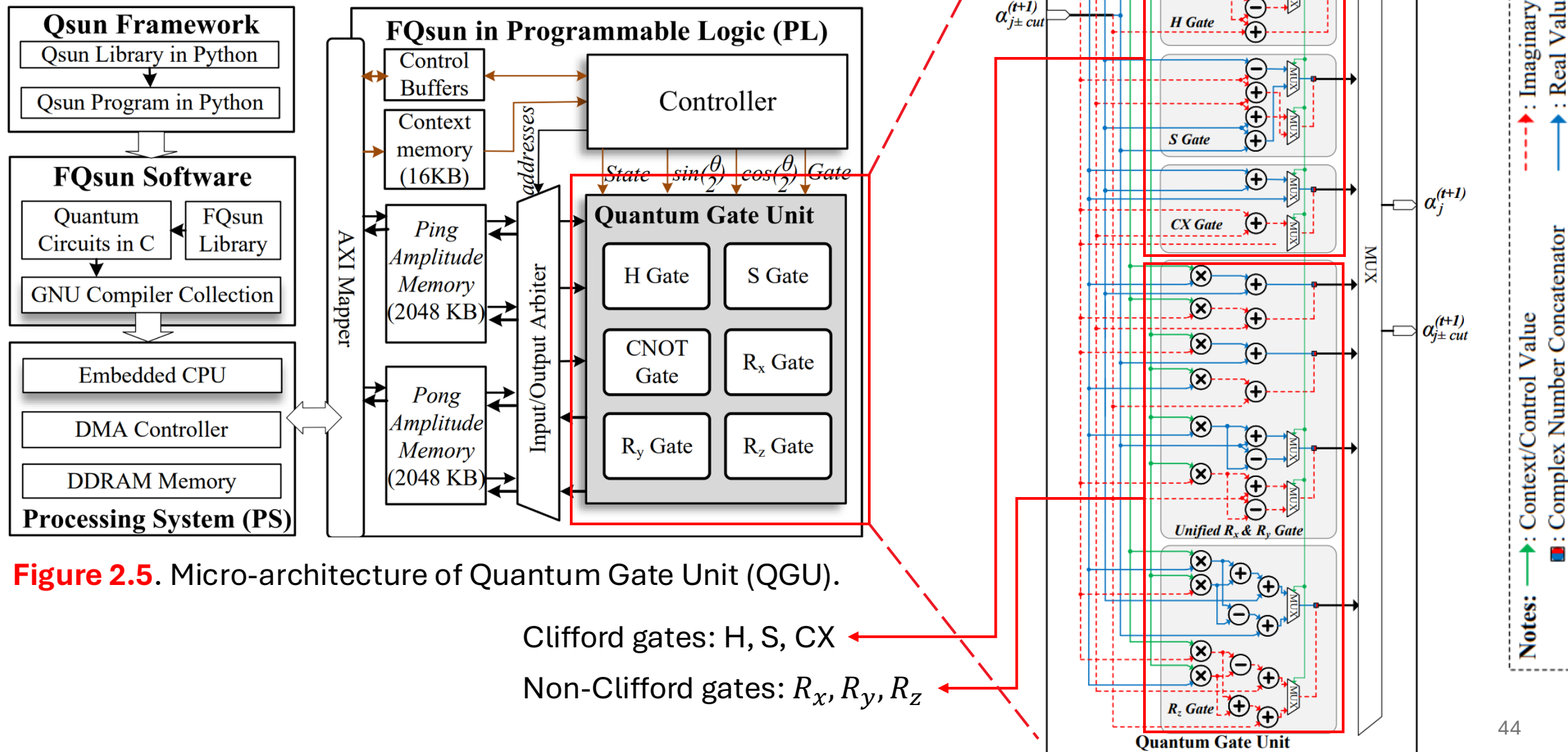


Figure 2.4. Timing chart of FQsun operation using Ping/Pong Amplitude Memories. 43

2.3.3. Idea #3: configurable Quantum Gate Unit (QGU)



2.3.4. #Idea 4: multiple precisions

Table 2.3. Utilization of FQSun on ZCU102 FPGA

Design	Name	LUTs	FFs	BRAM	DSP
FP16	Buffers & AXI Mapper	268	290	0	0
	Context Memory	4,428	0	0	0
	Ping & Pong Memories	3,071	36	464	0
	QGU	5,530	2,752	0	44
	FQsun Controller	937	973	0	0
	Total	14,234	4,051	464	44
FP32	Buffers & AXI Mapper	295	327	0	0
	Context Memory	6,556	0	0	0
	Ping & Pong Memories	3,796	36	456	0
	QGU	6,258	5,296	0	88
	FQsun Controller	1,188	1,372	0	0
	Total	18,093	7,031	456	88
FX16	Buffers & AXI Mapper	264	289	0	0
	Context Memory	4,387	0	0	0
	Ping & Pong Memories	3,168	36	464	0
	QGU	457	142	0	14
	FQsun Controller	990	973	0	0
	Total	9,266	1,440	464	14
FX24	Buffers & AXI Mapper	272	307	0	0
	Context Memory	5,722	0	0	0
	Ping & Pong Memories	3,086	24	344	0
	QGU	684	206	0	28
	FQsun Controller	939	828	0	0
	Total	10,703	1,635	344	28
FX32	Buffers & AXI Mapper	297	327	0	0
	Context Memory	6,888	0	0	0
	Ping & Pong Memories	3,934	36	456	0
	QGU	914	326	0	56
	FQsun Controller	1,136	1,372	0	0
	Total	13,169	2,061	456	56

Table 2.4. Basic gate's period and #Cycles per gate on FQsun

Version	Period (ns)	#Cycles per Gate					
		H	S	CX	R_x	R_y	R_z
FP16	6.66	6	4	4	6	6	8
FP32	7.35	6	4	4	6	6	8
FX16	7.35	4	2	2	4	4	4
FX24	8.00	4	2	2	4	4	4
FX32	9.35	4	2	2	4	4	4

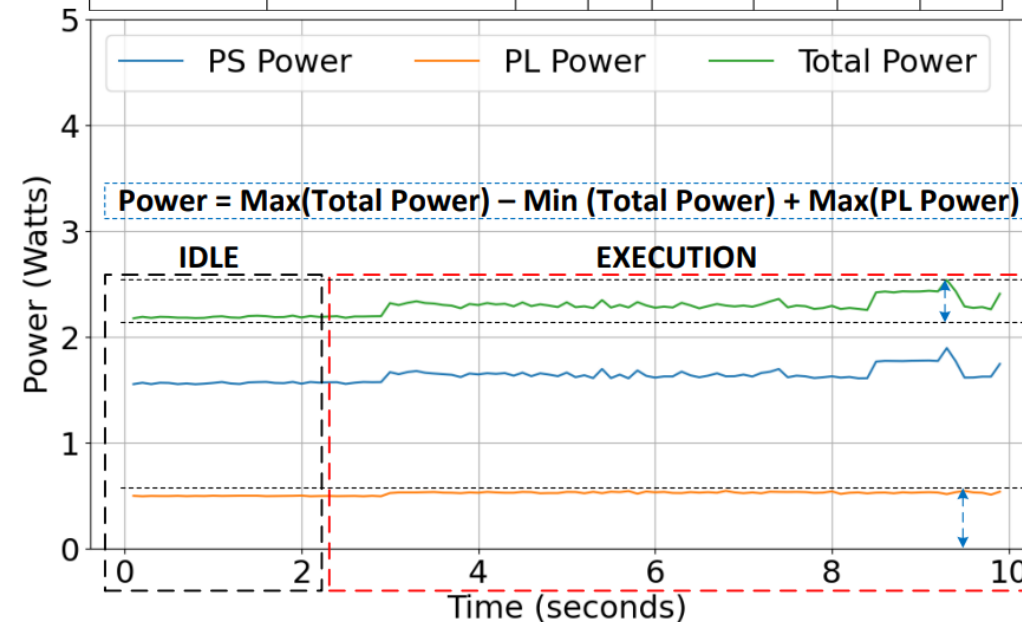


Figure 2.6. Power measurement of FQsun (FX32).

2.4. Benchmarking tasks

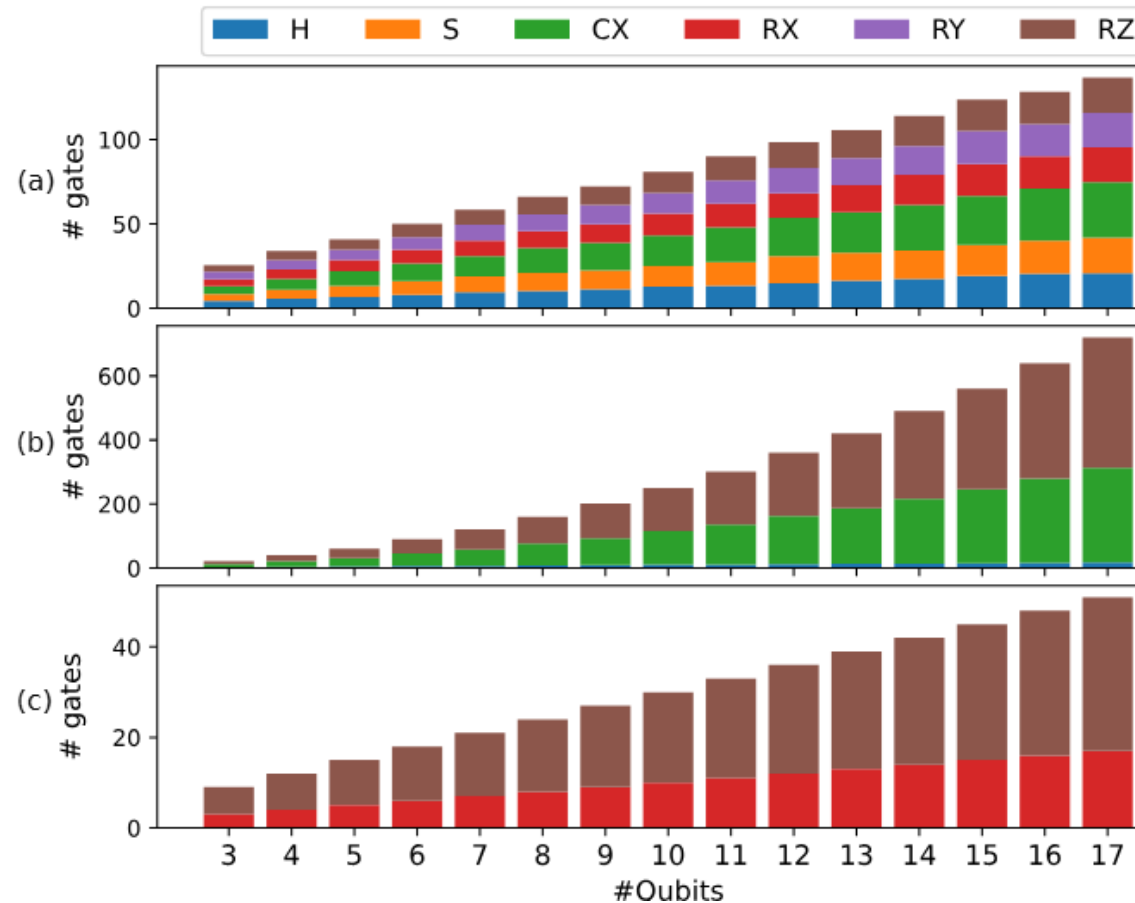


Figure 2.7. Gate distribution for
 (a) Random quantum circuit (RQC)
 (b) Quantum Fourier Transform (QFT) and
 (c) ZXZ with parameter-shift rule (PSR).

Randomize quantum circuit (RQC) with $\sim$ uniform gate's distribution

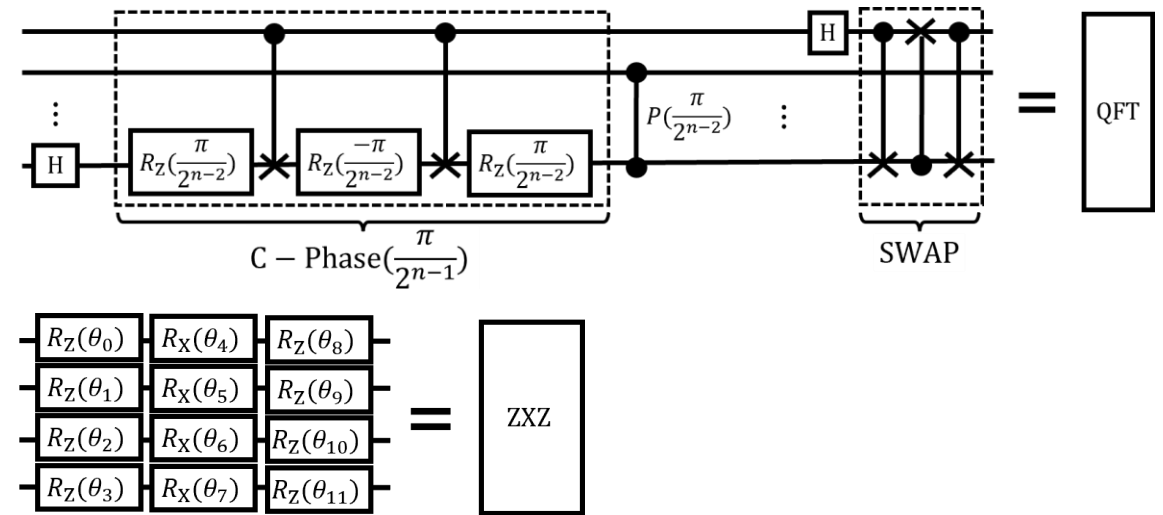


Figure 2.8. Circuit representation for
 (b) QFT and
 (c) PSR.

2.4.1. Result #1: accuracy metrics

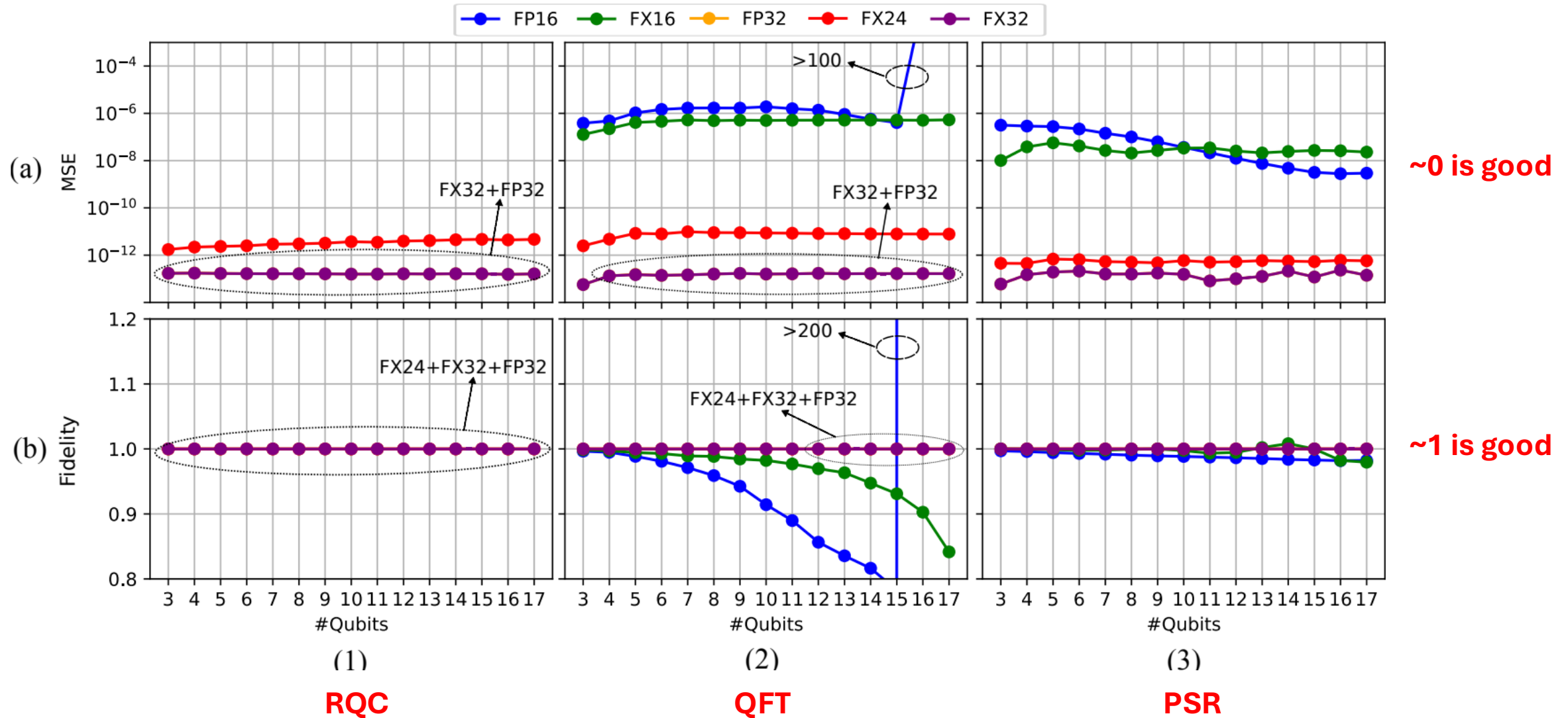
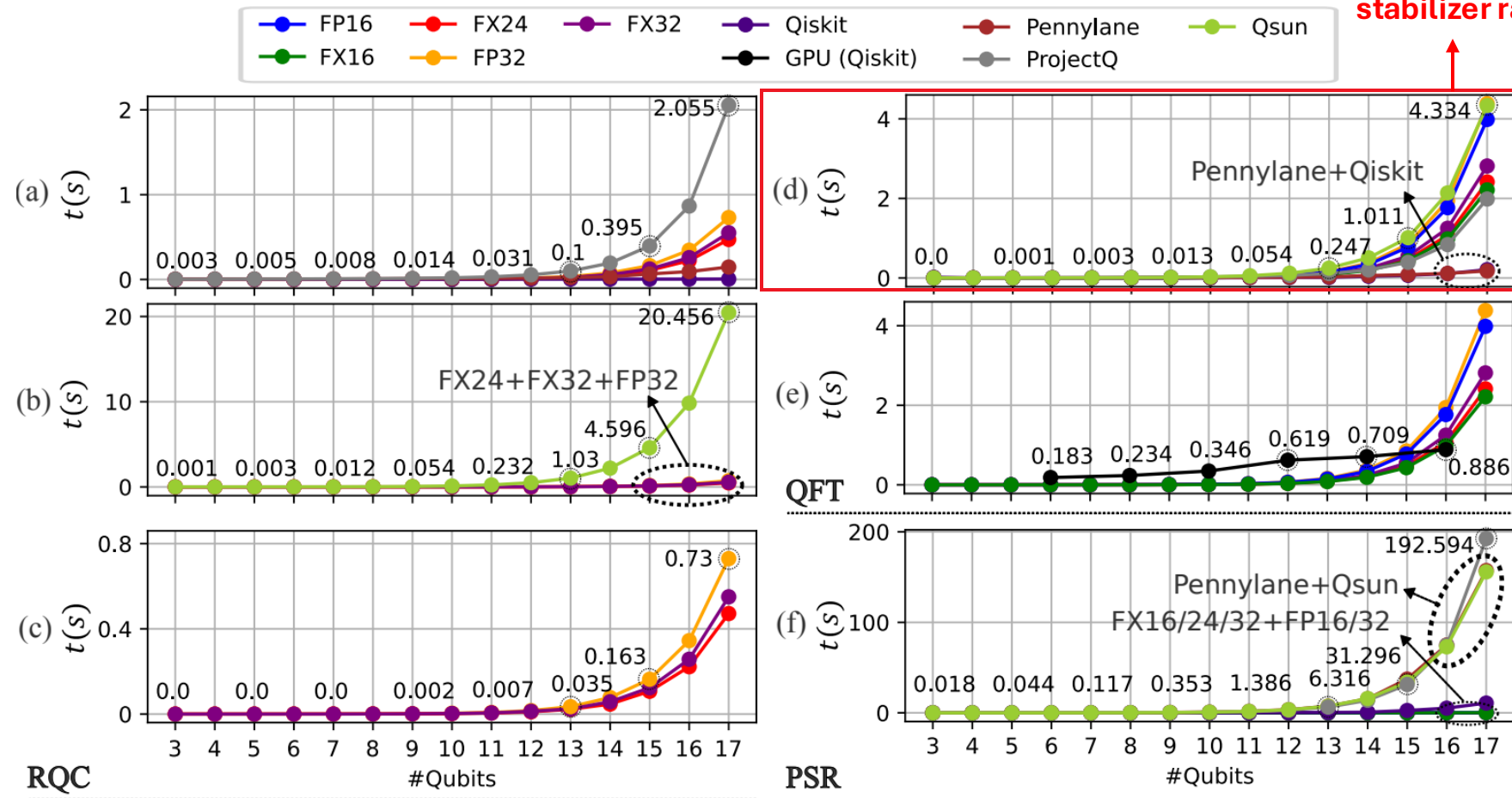


Figure 2.9. MSE/Fidelity between FQsun version. In all cases, FP32 and FX32 are overlapped. For QFT, the FP16 version performs the wrong results since it can not present any $\alpha_j \leq 1/2^{16} \approx 10^{-5}$.

2.4.2. Results #2: execution time & PDP



In this case,
Software
seems better
due to the
stabilizer rank

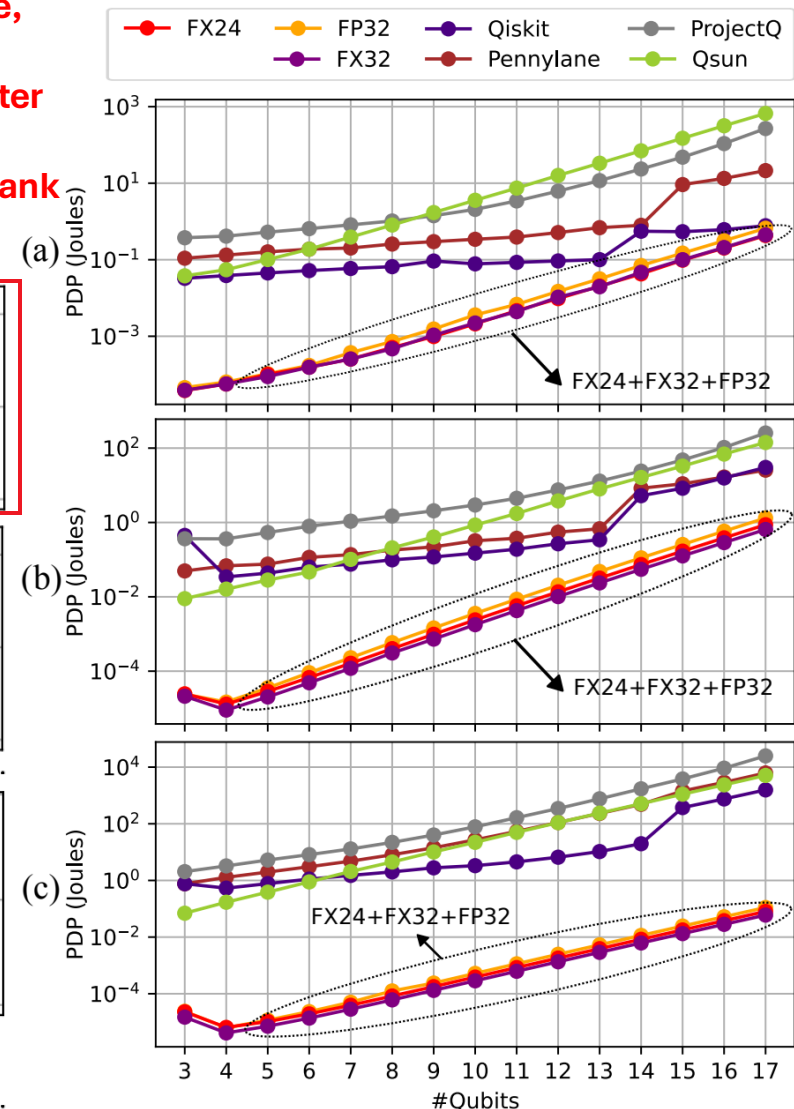


Figure 2.11. PDP through (a) RQC (b) QFT and (c) PSR.

Figure 2.10. The execution time on (a-c) RQC (d) QFT execution time comparison between Software + FQsun. (e) We compare GPU A100 and FQsun. (f) PSR execution time comparison between Software + FQsun.

2.4.4. Result #4: comparison between related works

Table 2.5. Comparative analysis of FQsun and existing FPGA-based emulators on QFT's performance.

Works	Device	Frequency (MHz)	Flexibility	Precision	#Qubits	Execution time (s)	#Gates [†]	NGS ^{††}
[57]	Xilinx XCVU9P	233	✗	18-bit FX	16	1.20×10^{-3}	-	-
[2]	Arria 10AX115N4F45E3SG	233	✓	32-bit FP	16	1.84×10^1	528	5.33×10^{-7} No flexibility
[59]	Xilinx XCKU115	160	✗	16-bit FX	16	2.70×10^{-1}	136	3.03×10^{-8}
[63]	AMD Xilinx Zynq-7000	100	✓	32-bit FX	6	1.15×10^{-4}	10	1.80×10^{-7}
[64]	2 × Intel Stratix 10 MX2100	299	✗	32-bit FP	30	4.47×10^0	465	8.95×10^{-12} No flexibility
This work	Xilinx ZCU102	150	✓	16-bit FP	18	8.90×10^0	810	4.19×10^{-8}
		136	✓	32-bit FP	17	4.30×10^0	721	4.55×10^{-8}
		136	✓	16-bit FX	18	4.90×10^0	810	2.31×10^{-8}
		125	✓	24-bit FX	17	2.41×10^0	721	2.52×10^{-8}
		125	✓	32-bit FX	17	2.81×10^0	721	2.97×10^{-8}

(†) The #Gate in this work is higher than other work due to the no-use of control-rotation gates.

(††) The Normalized Gate Speed (NGS) ($s / (\text{gate} \times \text{amplitude}) = \text{Execution time} / (\#Gates \times 2^{\#Qubits})$), smaller is better.